Bölüm 1: Giriş

Algoritmalar

Algoritma nedir? | Algoritma analizi | O, Ω ve Θ gösterimleri | Karmaşıklık sınıfları

Eklemeli sıralama | Birleştirme sıralaması | İkili arama ve üs alma | Öklid algoritması

Sercan KÜLCÜ · Giresun Üniversitesi · Bilgisayar Mühendisliği

GİRİŞ

Öğrenme Çıktıları

Algoritma

Algoritmanın tanımını ve özelliklerini (girdi, çıktı, açıklık, sonluluk, etkinlik) açıklamak; günlük bir işi algoritma olarak yazmak.

Analiz

Çalışma süresini girdi boyutu n'nin fonksiyonu T(n) olarak ifade etmek; en kötü, ortalama ve en iyi durumu ayırt etmek.

Gösterimler

O, Ω ve Θ gösterimlerinin üst, alt ve sıkı sınır anlamını kavramak; bir koddan karmaşıklık sınıfını (1, log n, n, n log n, n^2, n^3, 2^n, n!) okumak.

Temel algoritmalar

Eklemeli sıralama, birleştirme, ikili arama, hızlı üs alma ve Öklid algoritmasını adım adım izlemek ve özyineleme bağıntılarını kurmak.

ALGORİTMA KAVRAMI

Algoritma Nedir?

- Bir problemi çözmek veya bir görevi yerine getirmek için izlenen **adım adım yönergeler dizisidir**.
- Girdi verileri alınır, işlemler yapılır, **çıktı** elde edilir.
- Her adım **belirli, açık ve anlaşılır** olmalı; algoritma **sınırlı sayıda adımda** sonlanmalıdır.

Girdi (input)
Sıfır veya daha fazla değer dışarıdan alınır.

Çıktı (output)
En az bir sonuç üretilir; sonuç girdiyle ilişkilidir.

Açıklık
Her adımın tek bir anlamı vardır; yoruma yer bırakmaz.

Sonluluk
Her girdi için sonlu sayıda adımdan sonra durur.

Etkinlik
Her adım temel ve gerçekten yapılabilir bir işlemdir.

Doğruluk
Her geçerli girdi için doğru çıktıyı verir.

ALGORİTMA KAVRAMI

Günlük Hayattan: Çay Hazırlama Algoritması

Girdi: su, çay yaprakları, şeker (isteğe bağlı), süt (isteğe bağlı)
Çıktı: çay

1. Suyu kaynat.
2. Kaynamış suyu demliğe koy.
3. Demliğin içine çay yapraklarını ekle.
4. Çayın demlenmesini bekle.
5. Demlenen çayı fincana süzerek koy.
6. İsteğe bağlı olarak şeker ve süt ekle.

Sıra önemlidir

Adım 2 ile 3'ün yeri değişebilir, ama 1 ile 5'in yeri değişemez: algoritma bir **sıralı** adımlar listesidir.

Açık mı?

"Demlenmesini bekle" ne kadar? Bir bilgisayar için adım şöyle olmalıdır: **10 dakika bekle**.

Karar

Adım 6 bir **koşul** içerir: "şeker isteniyorsa ekle". Algoritmalar karar noktaları da içerir.

Ortalama Bulma: Algoritmadan Koda

Algoritma

Girdi: Sayı1, Sayı2, Sayı3 · Çıktı: Ortalama

1. Toplam = Sayı1 + Sayı2 + Sayı3
2. Ortalama = Toplam / 3
3. Ortalama değerini ekrana yazdır.

Python

```python
def ortalama_hesapla(sayi1, sayi2, sayi3):
    ortalama = (sayi1 + sayi2 + sayi3) / 3
    return ortalama

sayi1 = float(input("Birinci sayıyı girin: "))
sayi2 = float(input("İkinci sayıyı girin: "))
sayi3 = float(input("Üçüncü sayıyı girin: "))
ort = ortalama_hesapla(sayi1, sayi2, sayi3)
print("Girilen sayıların ortalaması:", ort)
```

Java

```java
public static void main(String[] args) {
  Scanner giris = new Scanner(System.in);
  System.out.println("Birinci sayıyı girin: ");
  double sayi1 = giris.nextDouble();
  System.out.println("İkinci sayıyı girin: ");
  double sayi2 = giris.nextDouble();
  System.out.println("Üçüncü sayıyı girin: ");
  double sayi3 = giris.nextDouble();
  double ortalama = (sayi1 + sayi2 + sayi3) / 3;
  System.out.println("Sayıların ortalaması: "
                     + ortalama);
}
```

Aynı algoritma farklı dillerde yazılabilir: algoritma **dilden bağımsızdır**, program onun bir gerçekleştirimidir.

ALGORİTMA KAVRAMI

Algoritmalar Nerede? Başlıca Türler

Sıralama
Öğeleri bir düzene koyar: eklemeli sıralama, birleştirme sıralaması, hızlı sıralama.

Arama
Bir öğeyi bulur: doğrusal ve ikili arama, hash.

Çizge
En kısa yol (Dijkstra), minimum kapsayan ağaç, maksimum akış (Ford–Fulkerson).

Dizgi ve sıkıştırma
Metin arama, trie; dosya boyutunu küçültme: Huffman, gzip.

Tasarım teknikleri
Böl ve fethet, dinamik programlama, açgözlü, geri izleme, dal sınır.

Rastgele ve yaklaşık
Aynı girdide farklı yol izleyebilen **rastgele** algoritmalar; zor problemlere yaklaşık çözüm.

Deterministik algoritma aynı girdi için her zaman aynı adımları izler ve aynı çıktıyı verir (ör. en büyük öğeyi bulma). **Rastgele** (randomized) algoritma zar atar: hızlı sıralamada pivotu rastgele seçmek gibi.

Algoritma Analizi

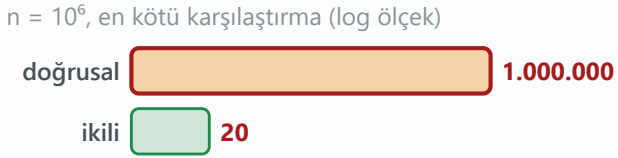
- Bir algoritmanın **performansını** anlamak ve değerlendirmek için kullanılır.
- İşlemci ve bellek gibi **kaynakların** verimli kullanılmasını sağlar.
- **Zaman analizi:** algoritmanın çalışma süresini ölçer; genellikle büyük O gösterimiyle yazılır.
- **Bellek analizi:** algoritmanın ek bellek kullanımını (alan karmaşıklığı) değerlendirir.
- Aynı problemi çözen algoritmalar arasında **seçim** yapmamızı sağlar.

Neden önemli?

10^6 öğeli sıralı bir dizide arama:
doğrusal arama → en kötü **1.000.000** karşılaştırma
ikili arama → en fazla **20** karşılaştırma

Bellek örnekleri

En büyük öğeyi bulma: **O(1)** ek bellek.
Birleştirme sıralaması: **O(n)** yardımcı dizi.
Özyineleme: çağrı yığını kadar bellek.



ALGORİTMA ANALİZİ

Zaman Karmaşıklığı

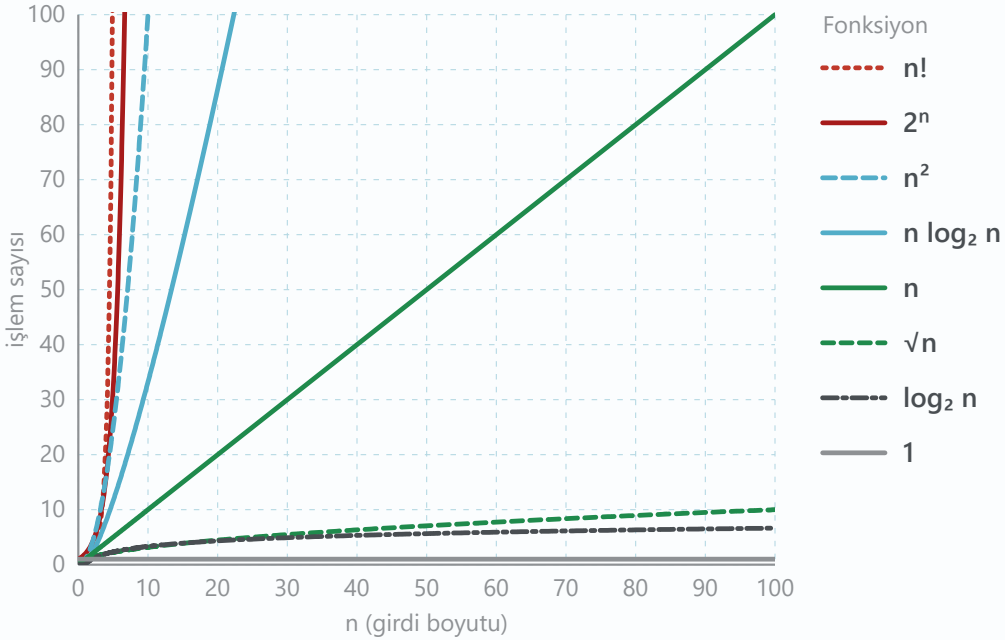
- Bir algoritmanın çalışma süresini **girdi boyutu n** ile ilişkilendiren kavramdır: **T(n)**.
- Saniye değil, **temel işlem sayısı** sayılır: saniye bilgisayara ve dile göre değişir, işlem sayısı değişmez.
- Veri büyüdükçe performansın **nasıl değiştiğini** inceler; bir algoritmanın iyi ya da kötü olduğunu belirlemede kritik faktördür.

Örnek: n öğeli dizinin toplamı, satır satır sayım

```
int toplam = 0;                    // 1 kez
for (int i = 0; i < n; i++)        // n+1 karşılaştırma
  toplam += dizi[i];               // n kez
return toplam;                     // 1 kez
```

T(n) = 1 + (n + 1) + n + 1 = **2n + 3**. Sabitler makineye bağlıdır; önemli olan T(n)'nin **n ile doğrusal** büyümesidir → Θ(n).

Fonksiyonların Büyüme Hızları



n = 10'da bile n! (3,6 milyon) ve 2^n (1024) grafiğin dışına çıkar; $\log_2 n$ ve $\sqrt{n}$ ise n = 100'de hâlâ 10'un altındadır.

Büyüme Tablosu: n Büyürken

f(n)	n = 10	n = 100	n = 1000	$n = 10^6$	süre ($n = 10^6$)
1	1	1	1	1	1 ns
$\log_2 n$	3,3	6,6	10,0	19,9	19,9 ns
$\sqrt{n}$	3,2	10	31,6	1.000	1 µs
n	10	100	1.000	1.000.000	1 ms
$n \log_2 n$	33,2	664,4	9.966	$2{,}0 \cdot 10^7$	19,9 ms
n^2	100	10.000	1.000.000	$1{,}0 \cdot 10^{12}$	16,7 dk
2^n	1.024	$1{,}3 \cdot 10^{30}$	$\approx 1{,}1 \cdot 10^{301}$	$\approx 10^{301030}$	evrenin yaşından çok
n!	3.628.800	$9{,}3 \cdot 10^{157}$	$\approx 10^{2567}$	$\approx 10^{5565708}$	evrenin yaşından çok

Son sütun: saniyede 10^9 temel işlem yapan bir bilgisayarda $n = 10^6$ için tahmini süre. 2^n ve n! için $n = 100$ bile ($\approx 10^{30}$ ve 10^{158} işlem) pratikte çözülemez.

Daha hızlı bir bilgisayar sabiti küçültür; **daha iyi bir algoritma büyüme hızını** değiştirir. n^2 yerine $n \log n$, 10^6 öğede dakikaları milisaniyelere indirir.

Deneyin: Büyüme Hesaplayıcı

n = 1000 | 10 | 100 | 10^4 | 10^6 | n × 2

f(n)	işlem sayısı	10^9 işlem/s ile süre	bağıl
1	1	1 ns	
$\log_2 n$	10	10 ns	
$\sqrt{n}$	31,6	31,6 ns	
n	1.000	1 µs	
$n \log_2 n$	9.965,8	10 µs	
n^2	1.000.000	1 ms	
n^3	$1 \cdot 10^9$	1 s	
2^n	$1{,}1 \cdot 10^{301}$	$\approx 10^{284}$ yıl	
n!	$4 \cdot 10^{2567}$	$\approx 10^{2551}$ yıl	

n = **1.000** · çubuk: işlem sayısının basamak sayısı (log ölçek) · kırmızı: 1 saatten uzun

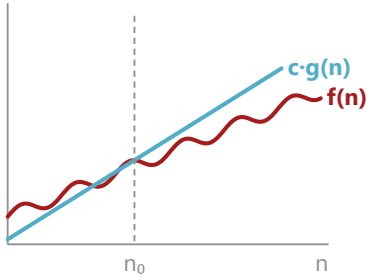
"n × 2" düğmesiyle girdiyi ikiye katlayın: n sütunu 2 katına, n^2 4 katına, n^3 8 katına çıkar; $\log_2 n$ yalnızca 1 artar, 2^n ise karesini alır.

ALGORİTMA ANALİZİ

Asimptotik Gösterimler: O, Ω, Θ

Büyük O: üst sınır

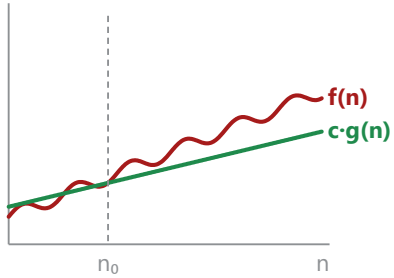
$f(n) = O(g(n))$: $n \geq n_0$ için $f(n) \leq c \cdot g(n)$. "En fazla bu kadar hızlı büyür."



Süre 100 s ile sınırlıysa daha kısa sürebilir, ama **daha uzun süremez**.

Omega Ω: alt sınır

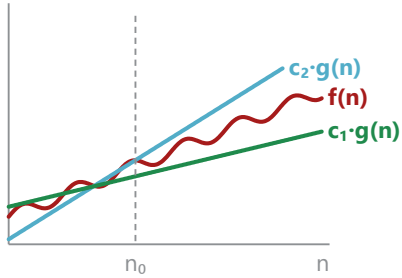
$f(n) = \Omega(g(n))$: $n \geq n_0$ için $f(n) \geq c \cdot g(n)$. "En az bu kadar hızlı büyür."



Alt sınır 100 s ise daha uzun sürebilir, ama **daha kısa süremez**.

Theta Θ: sıkı sınır

$f(n) = \Theta(g(n))$: hem $O(g(n))$ hem $\Omega(g(n))$. "Tam olarak bu hızda büyür."



Doğrusal aramanın en kötü durumu: $c_1 \cdot n \leq T(n) \leq c_2 \cdot n \rightarrow$ **Θ(n)**.

Dikkat: O, Ω, Θ **sınır** türleridir; en kötü / en iyi **durum** ile aynı şey değildir. Uygulamada O çoğunlukla en kötü durum süresi için kullanılır.

KARMAŞIKLIK ÖRNEKLERİ

Doğrusal Arama (Linear Search)

```java
int dogrusalAra(int[] dizi, int aranan) {
  for (int i = 0; i < dizi.length; i++) {
    if (dizi[i] == aranan) {
      return i;  // bulundu!
    }
  }
  return -1;  // bulunamadı
}
```

Zaman karmaşıklığı

En kötü durumda (hedef yok ya da sonda) döngü n kez döner:
$T(n) = c_1 \cdot n + c_2$
c_1 ve c_2 bilgisayarın hızı, programlama dili gibi etkenlere bağlıdır.

Θ(n)

Süre, girdi boyutu n ile **orantılı** büyür: yeterince büyük n için bazı k_1, $k_2 > 0$ sabitleriyle
$k_1 \cdot n \le T(n) \le k_2 \cdot n$.
En iyi durum: hedef ilk öğe → 1 karşılaştırma, Θ(1).

KARMAŞIKLIK ÖRNEKLERİ

Sabit ve Doğrusal Zaman: O(1), O(n)

İlk öğeyi yazdırma T(n) = O(1)

```
void ilkElemaniYazdir(int[] dizi) {
  if (dizi.length > 0)
    System.out.println("İlk eleman: "
                       + dizi[0]);
  else
    System.out.println("Dizi boş");
}
```

Dizi 10 öğeli da olsa 10 milyon öğeli da olsa **tek** karşılaştırma ve tek erişim yapılır: süre n'den bağımsızdır.

En büyük öğeyi bulma T(n) = O(n)

```
Integer enBuyukBul(int[] dizi) {
  if (dizi.length == 0)
    return null;              // dizi boşsa
  int enBuyuk = dizi[0];
  for (int eleman : dizi)
    if (eleman > enBuyuk)
      enBuyuk = eleman;
  return enBuyuk;
}
```

Her öğe **bir kez** incelenir: n karşılaştırma. Ek bellek O(1). Örnek: [5, 8, 12, 4] → 12.



Logaritmik Zaman: İkili Arama, O(log n)

```
int ikiliArama(int[] dizi, int aranan) {
  int alt = 0, ust = dizi.length - 1;
  while (alt <= ust) {
    int orta = (alt + ust) / 2;
    int tahmin = dizi[orta];
    if (tahmin == aranan)
      return orta;     // bulundu
    else if (tahmin < aranan)
      alt = orta + 1;
    else
      ust = orta - 1;
  }
  return -1;          // bulunamadı
}
```

Dizi **sıralı** olmalıdır. Her adımda aranan aralık **yarıya** iner: n → n/2 → n/4 → … → 1.

n	en fazla adım
8	4
1.000	10
1.000.000	20
$1{,}0 \cdot 10^9$	30

En fazla $\lfloor \log_2 n \rfloor + 1$ adım → **T(n) = O(log n)**. Adım adım örnek: Böl ve Fethet bölümünde.

KARMAŞIKLIK ÖRNEKLERİ

Asal Sayı Kontrolü: O(√n)

```java
boolean asalMi(int n) {
  if (n <= 1)
    return false;
  else if (n == 2)
    return true;
  else if (n % 2 == 0)
    return false;
  int kokN = (int) Math.sqrt(n) + 1;
  for (int i = 3; i < kokN; i += 2)
    if (n % i == 0)
      return false;
  return true;
}
```

Neden √n'e kadar?

n = a · b ise a ve b'den biri **√n'den küçük ya da eşittir**. √n'e kadar bölen yoksa n asaldır.

Kaç bölme?

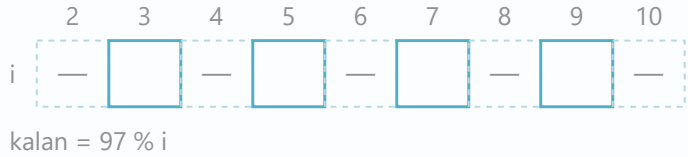
Çift sayılar baştan elenir; yalnızca tek i denenir: en fazla ≈ √n / 2 bölme → **T(n) = O(√n)**.

n = 97: i = 3, 5, 7, 9 → **4** bölme (2'den 96'ya denense 95).
n = 1.000.000.007: **15.810** bölme.

Not: burada girdi boyutu n sayısının **değeridir**. Sayının bit uzunluğu b ise $\sqrt{n} = 2^{b/2}$: bu yöntem bit sayısına göre üstel büyür.

asalMi(97): Adım Adım

```
1  boolean asalMi(int n) {
▶    if (n <= 1)
3      return false;
▶    else if (n == 2)
5      return true;
▶    else if (n % 2 == 0)
7      return false;
8    int kokN = (int) Math.sqrt(n) + 1;
9    for (int i = 3; i < kokN; i += 2)
10     if (n % i == 0)
11       return false;
12   return true;
13 }
```



çift i'ler (gri) hiç denenmez

n = 97: 1'den büyük, 2'ye eşit değil ve tek. İlk üç kontrolün hiçbiri sonuç döndürmez; 2'nin katları baştan elenmiş olur.

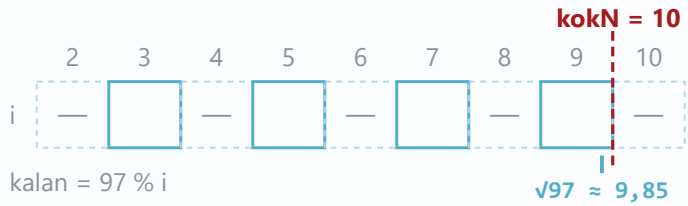
Adım 1 / 8

DEĞİŞKENLER

`n = 97` `kokN = –` `i = –`

asalMi(97): Adım Adım

```
1  boolean asalMi(int n) {
2    if (n <= 1)
3      return false;
4    else if (n == 2)
5      return true;
6    else if (n % 2 == 0)
7      return false;
▶    int kokN = (int) Math.sqrt(n) + 1;
9    for (int i = 3; i < kokN; i += 2)
10     if (n % i == 0)
11       return false;
12   return true;
13 }
```



çift i'ler (gri) hiç denenmez

kokN = (int) √97 + 1 = 9 + 1 = **10**. Yalnızca 10'dan küçük tek sayılar denenecek: 3, 5, 7, 9.

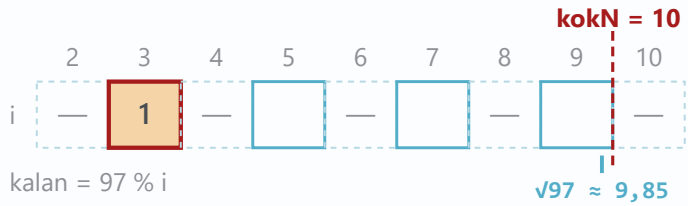
Adım 2 / 8

DEĞİŞKENLER

`n = 97` `kokN = 10` `i = –`

asalMi(97): Adım Adım

```
1  boolean asalMi(int n) {
2    if (n <= 1)
3      return false;
4    else if (n == 2)
5      return true;
6    else if (n % 2 == 0)
7      return false;
8    int kokN = (int) Math.sqrt(n) + 1;
▶    for (int i = 3; i < kokN; i += 2)
▶      if (n % i == 0)
11       return false;
12   return true;
13 }
```



çift i'ler (gri) hiç denenmez

i = 3: 97 % 3 = **1** ≠ 0 → 3, 97'yi bölmez. `i += 2`.

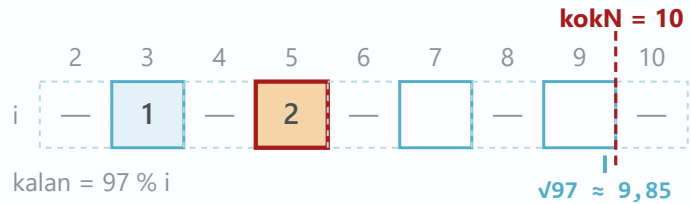
Adım 3 / 8

DEĞİŞKENLER

`n = 97` `kokN = 10` `i = 3` `n % i = 1`

asalMi(97): Adım Adım

```
1  boolean asalMi(int n) {
2    if (n <= 1)
3      return false;
4    else if (n == 2)
5      return true;
6    else if (n % 2 == 0)
7      return false;
8    int kokN = (int) Math.sqrt(n) + 1;
▶    for (int i = 3; i < kokN; i += 2)
▶      if (n % i == 0)
11       return false;
12   return true;
13 }
```



çift i'ler (gri) hiç denenmez

i = 5: 97 % 5 = **2** ≠ 0 → 5, 97'yi bölmez. `i += 2`.

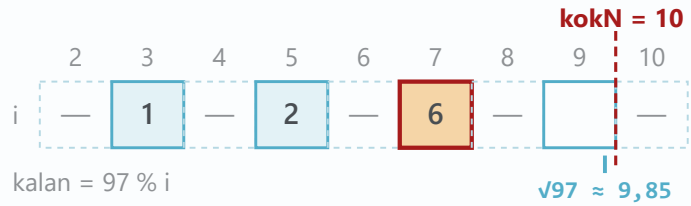
Adım 4 / 8

DEĞİŞKENLER

`n = 97` `kokN = 10` `i = 5` `n % i = 2`

asalMi(97): Adım Adım

```
1  boolean asalMi(int n) {
2    if (n <= 1)
3      return false;
4    else if (n == 2)
5      return true;
6    else if (n % 2 == 0)
7      return false;
8    int kokN = (int) Math.sqrt(n) + 1;
▶    for (int i = 3; i < kokN; i += 2)
▶      if (n % i == 0)
11       return false;
12   return true;
13 }
```



çift i'ler (gri) hiç denenmez

i = 7: 97 % 7 = **6** ≠ 0 → 7, 97'yi bölmez. `i += 2`.

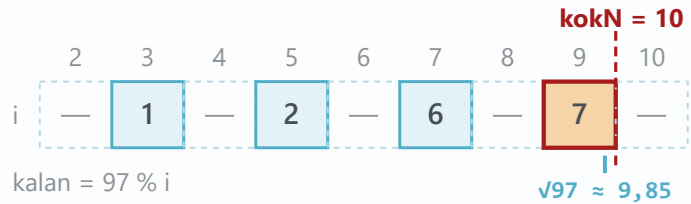
Adım 5 / 8

DEĞİŞKENLER

`n = 97` `kokN = 10` `i = 7` `n % i = 6`

asalMi(97): Adım Adım

```
1  boolean asalMi(int n) {
2    if (n <= 1)
3      return false;
4    else if (n == 2)
5      return true;
6    else if (n % 2 == 0)
7      return false;
8    int kokN = (int) Math.sqrt(n) + 1;
▶    for (int i = 3; i < kokN; i += 2)
▶      if (n % i == 0)
11       return false;
12   return true;
13 }
```



çift i'ler (gri) hiç denenmez

i = 9: 97 % 9 = **7** ≠ 0 → 9, 97'yi bölmez. `i += 2`. Sıradaki aday i = 11.

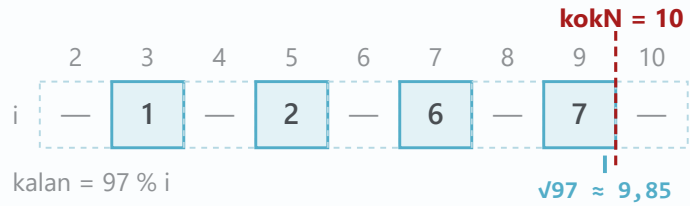
Adım 6 / 8

DEĞİŞKENLER

`n = 97` `kokN = 10` `i = 9` `n % i = 7`

asalMi(97): Adım Adım

```
1  boolean asalMi(int n) {
2    if (n <= 1)
3      return false;
4    else if (n == 2)
5      return true;
6    else if (n % 2 == 0)
7      return false;
8    int kokN = (int) Math.sqrt(n) + 1;
▶    for (int i = 3; i < kokN; i += 2)
10     if (n % i == 0)
11       return false;
12   return true;
13 }
```



çift i'ler (gri) hiç denenmez

i = 11 < 10 sağlanmaz, döngü biter. √97'ye kadar hiçbir tek sayı 97'yi bölmedi.

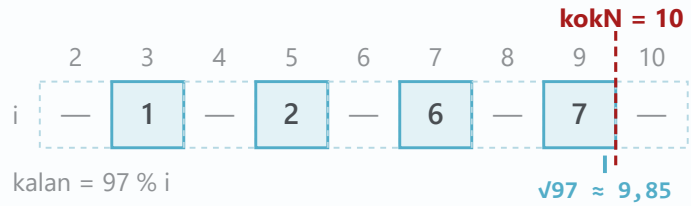
Adım 7 / 8

DEĞİŞKENLER

`n = 97` `kokN = 10` `i = 11` `n % i = –`

asalMi(97): Adım Adım

```
1  boolean asalMi(int n) {
2    if (n <= 1)
3      return false;
4    else if (n == 2)
5      return true;
6    else if (n % 2 == 0)
7      return false;
8    int kokN = (int) Math.sqrt(n) + 1;
9    for (int i = 3; i < kokN; i += 2)
10     if (n % i == 0)
11       return false;
▶    return true;
13 }
```



çift i'ler (gri) hiç denenmez

`return true` : 97 asaldır. Yalnızca **4 bölme** yetti; 2'den 96'ya kadar her sayı denenseydi 95 bölme gerekirdi.

Adım 8 / 8

DEĞİŞKENLER

`n = 97` `kokN = 10` `i = 11` `sonuç = true`

n log n Zaman: Çabuk (Hızlı) Sıralama

```
List<Integer> hizliSirala(List<Integer> liste) {
  if (liste.size() <= 1)
    return liste;
  int pivot = liste.get(0);
  List<Integer> kucukler = new ArrayList<>();
  List<Integer> buyukler = new ArrayList<>();
  for (int x : liste.subList(1, liste.size()))
    if (x <= pivot) kucukler.add(x);
    else buyukler.add(x);
  List<Integer> sonuc = hizliSirala(kucukler);
  sonuc.add(pivot);
  sonuc.addAll(hizliSirala(buyukler));
  return sonuc;
}
```

İlk öğe **pivot**; küçük-eşitler solda, büyükler sağda özyinelemeli sıralanır ve birleştirilir.

Ortalama: O(n log n)

Pivot diziyi yaklaşık ikiye bölerse $\log_2 n$ düzey olur, her düzeyde toplam n karşılaştırma yapılır.

En kötü: O(n^2)

Dizi zaten sıralıysa ilk öğe hep en küçüktür; bölünme $n-1$ / 0 olur: $(n-1) + (n-2) + \ldots + 1 = n(n-1)/2$.

Hızlı sıralamanın O(n log n) süresi **ortalama** durumdur; rastgele pivot seçimi en kötü durumu olası olmaktan çıkarır.

Karesel Zaman: Kabarcık Sıralaması, $O(n^2)$

```
void kabarcikSirala(int[] dizi) {
  int n = dizi.length;
  for (int i = 0; i < n; i++)
    for (int j = 0; j < n - i - 1; j++)
      if (dizi[j] > dizi[j + 1]) {
        int gecici = dizi[j];    // takas
        dizi[j] = dizi[j + 1];
        dizi[j + 1] = gecici;
      }
}
```

Komşu öğeler karşılaştırılır, sırasızsa takas edilir. Her dış turda en büyük öğe sona "kabarcık gibi" çıkar.

Sayım

i. turda iç döngü n − i − 1 kez döner: (n−1) + (n−2) + … + 1 + 0 = **n(n−1)/2** karşılaştırma.

n = 1000 → 499.500 karşılaştırma; n iki katına çıkınca süre ≈ **4 katına** çıkar. **$T(n) = O(n^2)$**.

Kübik Zaman: Matris Çarpımı, $\Theta(n^3)$

```
int[][] matrisCarp(int[][] A, int[][] B) {
  int n = A.length, m = B[0].length, p = B.length;
  int[][] sonuc = new int[n][m];
  for (int i = 0; i < n; i++)
    for (int j = 0; j < m; j++) {
      int eleman = 0;
      for (int k = 0; k < p; k++)
        eleman += A[i][k] * B[k][j];
      sonuc[i][j] = eleman;
    }
  return sonuc;
}
```

Girdi: A = $[a_{ij}]$, B = $[b_{ij}]$ (n × n). **Çıktı:** C = $[c_{ij}]$ = A · B, i, j = 1, 2, …, n.

Tanım

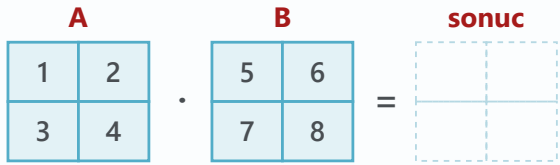
$$c_{ij} = \Sigma_{k=1..n}\, a_{ik} \cdot b_{kj}$$

1	2	·	5	6	=	19	22
3	4		7	8		43	50

c_{11} = 1·5 + 2·7 = 19. Her c_{ij} için n çarpma, n^2 öğe → **n^3 çarpma**: T(n) = $\Theta(n^3)$.

matrisCarp: 2 × 2 Matrisler Adım Adım

```
1  int[][] matrisCarp(int[][] A, int[][] B) {
▶    int n = A.length, m = B[0].length, p =
   B.length;
▶    int[][] sonuc = new int[n][m];
4    for (int i = 0; i < n; i++)
5      for (int j = 0; j < m; j++) {
6        int eleman = 0;
7        for (int k = 0; k < p; k++)
8          eleman += A[i][k] * B[k][j];
9        sonuc[i][j] = eleman;
10     }
11   return sonuc;
12 }
```



A ve B 2 × 2: n = m = p = 2. `sonuc` 2 × 2 sıfır matrisi olarak oluşturulur.

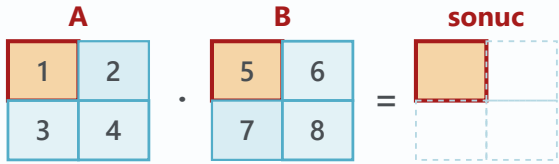
Adım 1 / 10

DEĞİŞKENLER

`n = 2` `m = 2` `p = 2` `i = –` `j = –` `k = –` `eleman = –` `çarpma = 0`

matrisCarp: 2 × 2 Matrisler Adım Adım

```
1  int[][] matrisCarp(int[][] A, int[][] B) {
2    int n = A.length, m = B[0].length, p =
   B.length;
3    int[][] sonuc = new int[n][m];
4    for (int i = 0; i < n; i++)
5      for (int j = 0; j < m; j++) {
▶        int eleman = 0;
▶        for (int k = 0; k < p; k++)
▶          eleman += A[i][k] * B[k][j];
9        sonuc[i][j] = eleman;
10     }
11   return sonuc;
12 }
```



eleman = 1·5 = 5

i = 0, j = 0: `eleman = 0`. k = 0: A[0][0] · B[0][0] = 1 · 5 = 5 eklenir.

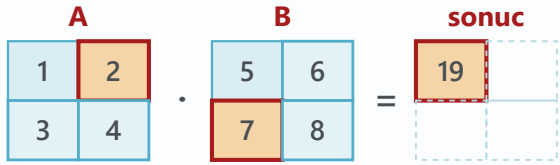
Adım 2 / 10

DEĞİŞKENLER

`n = 2` `m = 2` `p = 2` `i = 0` `j = 0` `k = 0` `eleman = 5` `çarpma = 1`

matrisCarp: 2 × 2 Matrisler Adım Adım

```
1  int[][] matrisCarp(int[][] A, int[][] B) {
2    int n = A.length, m = B[0].length, p =
   B.length;
3    int[][] sonuc = new int[n][m];
4    for (int i = 0; i < n; i++)
5      for (int j = 0; j < m; j++) {
6        int eleman = 0;
▶        for (int k = 0; k < p; k++)
▶          eleman += A[i][k] * B[k][j];
▶        sonuc[i][j] = eleman;
10     }
11   return sonuc;
12 }
```



eleman = 5 + 2·7 = 19

k = 1: A[0][1] · B[1][0] = 2 · 7 → eleman = 5 + 14 = **19**. İç döngü biter: `sonuc[0][0] = 19`.

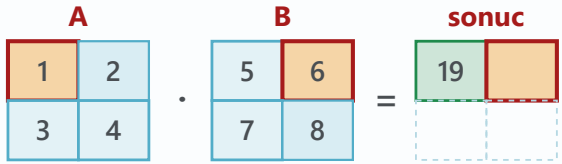
Adım 3 / 10

DEĞİŞKENLER

`n = 2` `m = 2` `p = 2` `i = 0` `j = 0` `k = 1` `eleman = 19` `çarpma = 2`

matrisCarp: 2 × 2 Matrisler Adım Adım

```
1  int[][] matrisCarp(int[][] A, int[][] B) {
2    int n = A.length, m = B[0].length, p =
   B.length;
3    int[][] sonuc = new int[n][m];
4    for (int i = 0; i < n; i++)
5      for (int j = 0; j < m; j++) {
▶        int eleman = 0;
▶        for (int k = 0; k < p; k++)
▶          eleman += A[i][k] * B[k][j];
9        sonuc[i][j] = eleman;
10     }
11   return sonuc;
12 }
```



eleman = 1·6 = 6

i = 0, j = 1: `eleman = 0` . k = 0: A[0][0] · B[0][1] = 1 · 6 = 6 eklenir.

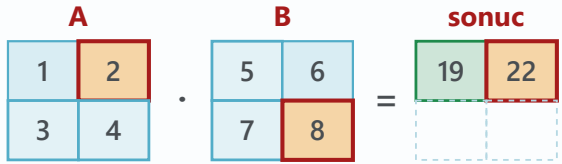
Adım 4 / 10

DEĞİŞKENLER

`n = 2` `m = 2` `p = 2` `i = 0` `j = 1` `k = 0` `eleman = 6` `çarpma = 3`

matrisCarp: 2 × 2 Matrisler Adım Adım

```
1  int[][] matrisCarp(int[][] A, int[][] B) {
2    int n = A.length, m = B[0].length, p =
   B.length;
3    int[][] sonuc = new int[n][m];
4    for (int i = 0; i < n; i++)
5      for (int j = 0; j < m; j++) {
6        int eleman = 0;
▶        for (int k = 0; k < p; k++)
▶          eleman += A[i][k] * B[k][j];
▶        sonuc[i][j] = eleman;
10     }
11   return sonuc;
12 }
```



eleman = 6 + 2·8 = 22

k = 1: A[0][1] · B[1][1] = 2 · 8 → eleman = 6 + 16 = **22**. İç döngü biter: `sonuc[0][1] = 22`.

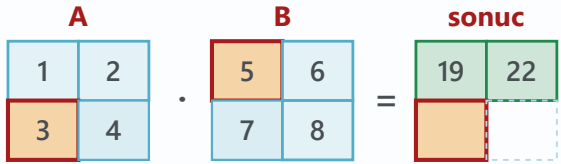
Adım 5 / 10

DEĞİŞKENLER

`n = 2` `m = 2` `p = 2` `i = 0` `j = 1` `k = 1` `eleman = 22` `çarpma = 4`

matrisCarp: 2 × 2 Matrisler Adım Adım

```
1  int[][] matrisCarp(int[][] A, int[][] B) {
2    int n = A.length, m = B[0].length, p =
   B.length;
3    int[][] sonuc = new int[n][m];
4    for (int i = 0; i < n; i++)
5      for (int j = 0; j < m; j++) {
▶        int eleman = 0;
▶        for (int k = 0; k < p; k++)
▶          eleman += A[i][k] * B[k][j];
9        sonuc[i][j] = eleman;
10     }
11   return sonuc;
12 }
```



eleman = 3·5 = 15

i = 1, j = 0: `eleman = 0`. k = 0: A[1][0] · B[0][0] = 3 · 5 = 15 eklenir.

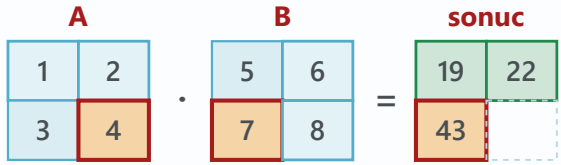
Adım 6 / 10

DEĞİŞKENLER

`n = 2` `m = 2` `p = 2` `i = 1` `j = 0` `k = 0` `eleman = 15` `çarpma = 5`

matrisCarp: 2 × 2 Matrisler Adım Adım

```
1  int[][] matrisCarp(int[][] A, int[][] B) {
2    int n = A.length, m = B[0].length, p =
   B.length;
3    int[][] sonuc = new int[n][m];
4    for (int i = 0; i < n; i++)
5      for (int j = 0; j < m; j++) {
6        int eleman = 0;
▶        for (int k = 0; k < p; k++)
▶          eleman += A[i][k] * B[k][j];
▶        sonuc[i][j] = eleman;
10     }
11   return sonuc;
12 }
```



eleman = 15 + 4·7 = 43

k = 1: A[1][1] · B[1][0] = 4 · 7 → eleman = 15 + 28 = **43**. İç döngü biter: `sonuc[1][0] = 43`.

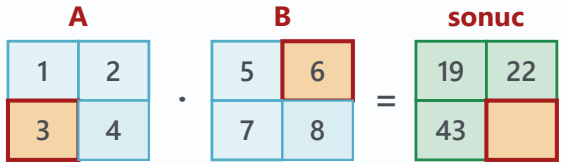
Adım 7 / 10

DEĞİŞKENLER

`n = 2` `m = 2` `p = 2` `i = 1` `j = 0` `k = 1` `eleman = 43` `çarpma = 6`

matrisCarp: 2 × 2 Matrisler Adım Adım

```
1  int[][] matrisCarp(int[][] A, int[][] B) {
2    int n = A.length, m = B[0].length, p =
   B.length;
3    int[][] sonuc = new int[n][m];
4    for (int i = 0; i < n; i++)
5      for (int j = 0; j < m; j++) {
▶        int eleman = 0;
▶        for (int k = 0; k < p; k++)
▶          eleman += A[i][k] * B[k][j];
9        sonuc[i][j] = eleman;
10     }
11   return sonuc;
12 }
```



eleman = 3·6 = 18

i = 1, j = 1: `eleman = 0`. k = 0: A[1][0] · B[0][1] = 3 · 6 = 18 eklenir.

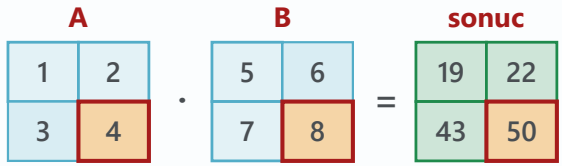
Adım 8 / 10

DEĞİŞKENLER

`n = 2` `m = 2` `p = 2` `i = 1` `j = 1` `k = 0` `eleman = 18` `çarpma = 7`

matrisCarp: 2 × 2 Matrisler Adım Adım

```
1  int[][] matrisCarp(int[][] A, int[][] B) {
2    int n = A.length, m = B[0].length, p =
   B.length;
3    int[][] sonuc = new int[n][m];
4    for (int i = 0; i < n; i++)
5      for (int j = 0; j < m; j++) {
6        int eleman = 0;
▶        for (int k = 0; k < p; k++)
▶          eleman += A[i][k] * B[k][j];
▶        sonuc[i][j] = eleman;
10     }
11   return sonuc;
12 }
```



eleman = 18 + 4·8 = 50

k = 1: A[1][1] · B[1][1] = 4 · 8 → eleman = 18 + 32 = **50**. İç döngü biter: `sonuc[1][1] = 50`.

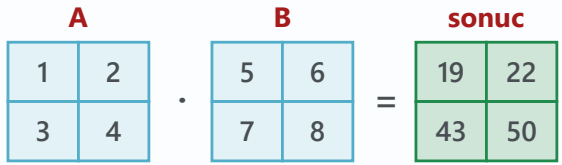
Adım 9 / 10

DEĞİŞKENLER

`n = 2` `m = 2` `p = 2` `i = 1` `j = 1` `k = 1` `eleman = 50` `çarpma = 8`

matrisCarp: 2 × 2 Matrisler Adım Adım

```
1  int[][] matrisCarp(int[][] A, int[][] B) {
2    int n = A.length, m = B[0].length, p =
   B.length;
3    int[][] sonuc = new int[n][m];
4    for (int i = 0; i < n; i++)
5      for (int j = 0; j < m; j++) {
6        int eleman = 0;
7        for (int k = 0; k < p; k++)
8          eleman += A[i][k] * B[k][j];
9        sonuc[i][j] = eleman;
10     }
▶    return sonuc;
12 }
```



sonuc = [[19, 22], [43, 50]] döner. **8 = 2^3 çarpma**: n^2 öğenin her biri için n çarpma → n^3.

Adım 10 / 10

DEĞİŞKENLER

`n = 2` `m = 2` `p = 2` `i = 2` `j = –` `k = –` `eleman = –` `çarpma = 8`

Faktöriyel Zaman: Tüm Permütasyonları Üretme

```java
void permutasyon(int[] dizi, int k) {
  if (k == dizi.length) {          // bir permütasyon hazır
    System.out.println(Arrays.toString(dizi));
    return;
  }
  for (int i = k; i < dizi.length; i++) {
    takas(dizi, k, i);             // i. elemanı k. konuma al
    permutasyon(dizi, k + 1);      // geri kalanları diz
    takas(dizi, k, i);             // eski hâline getir
  }
}
```

[1, 2, 3] → 123, 132, 213, 231, 321, 312. İlk konum için n, ikinci için n−1, … seçenek: **n · (n−1) · … · 1 = n!** permütasyon.

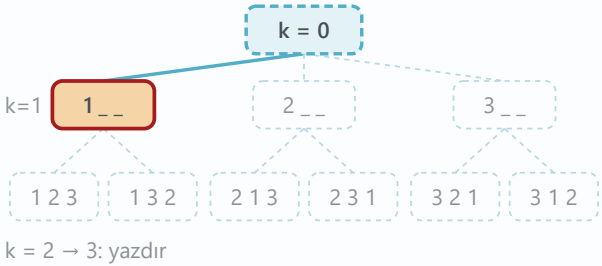
n	n!
3	6
5	120
10	3.628.800
15	$1{,}3 \cdot 10^{12}$
20	$2{,}4 \cdot 10^{18}$

Her permütasyonu yazmak n adım sürer: toplam **Θ(n · n!)**, kısaca "O(n!) sınıfı". n = 20 bile pratikte imkânsızdır.

permutasyon([1, 2, 3], 0): Adım Adım

```
1  void permutasyon(int[] dizi, int k) {
2    if (k == dizi.length) {
3      System.out.println(Arrays.toString(dizi));
4      return;
5    }
▶    for (int i = k; i < dizi.length; i++) {
▶      takas(dizi, k, i);
8      permutasyon(dizi, k + 1);
9      takas(dizi, k, i);
10   }
11 }
```

ÇIKTI



`permutasyon(dizi, 0)` : k = 0 < 3. i = 0: `takas(dizi, 0, 0)` diziyi değiştirmez; ilk konuma 1 sabitlenir.

Adım 1 / 10

DEĞİŞKENLER

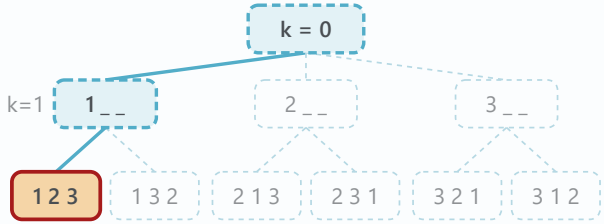
`k = 0` `i = 0` `dizi = [1, 2, 3]`

permutasyon([1, 2, 3], 0): Adım Adım

```
1  void permutasyon(int[] dizi, int k) {
▶    if (k == dizi.length) {
▶      System.out.println(Arrays.toString(dizi));
4      return;
5    }
6    for (int i = k; i < dizi.length; i++) {
7      takas(dizi, k, i);
8      permutasyon(dizi, k + 1);
9      takas(dizi, k, i);
10   }
11 }
```

ÇIKTI

```
[1, 2, 3]
```



k = 2 → 3: yazdır

k = 1 ve k = 2'de i = k takasları da diziyi değiştirmez. k == 3: ilk permütasyon **[1, 2, 3]** yazdırılır.

Adım 2 / 10

DEĞİŞKENLER

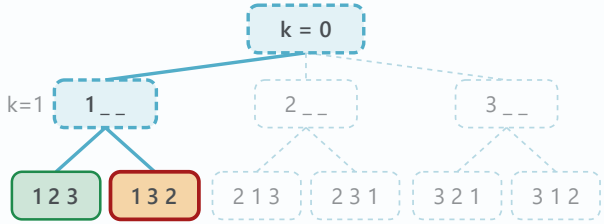
`k = 3` `i = –` `dizi = [1, 2, 3]`

permutasyon([1, 2, 3], 0): Adım Adım

```
1  void permutasyon(int[] dizi, int k) {
▶    if (k == dizi.length) {
▶      System.out.println(Arrays.toString(dizi));
4      return;
5    }
6    for (int i = k; i < dizi.length; i++) {
7      takas(dizi, k, i);
8      permutasyon(dizi, k + 1);
9      takas(dizi, k, i);
10   }
11 }
```

ÇIKTI

```
[1, 2, 3]
[1, 3, 2]
```



k = 2 → 3: yazdır

k = 1 düzeyinde önceki takas geri alındı; i = 2: `takas(dizi, 1, 2)`, ardından k = 2'de tek seçenek kalır. k == 3: ikinci permütasyon **[1, 3, 2]** yazdırılır.

Adım 3 / 10

DEĞİŞKENLER

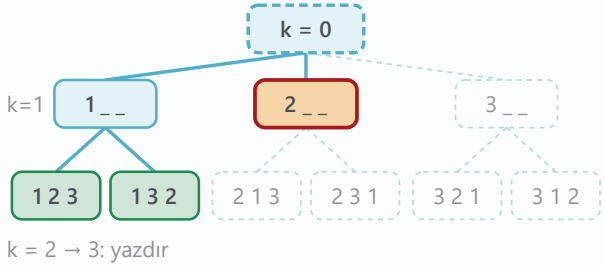
`k = 3` `i = –` `dizi = [1, 3, 2]`

permutasyon([1, 2, 3], 0): Adım Adım

```
1  void permutasyon(int[] dizi, int k) {
2    if (k == dizi.length) {
3      System.out.println(Arrays.toString(dizi));
4      return;
5    }
▶    for (int i = k; i < dizi.length; i++) {
▶      takas(dizi, k, i);
8      permutasyon(dizi, k + 1);
▶      takas(dizi, k, i);
10   }
11 }
```

ÇIKTI

```
[1, 2, 3]
[1, 3, 2]
```



k = 2 → 3: yazdır

Geri dönüşte `takas(dizi, 0, 0)` dizinin eski hâlini ([1, 2, 3]) geri getirir. i = 1: `takas(dizi, 0, 1)` → [2, 1, 3]; ilk konumda artık 2 var.

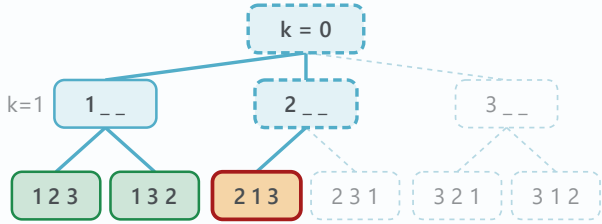
Adım 4 / 10

DEĞİŞKENLER

`k = 0` `i = 1` `dizi = [2, 1, 3]`

permutasyon([1, 2, 3], 0): Adım Adım

```
1  void permutasyon(int[] dizi, int k) {
▶    if (k == dizi.length) {
▶      System.out.println(Arrays.toString(dizi));
4      return;
5    }
6    for (int i = k; i < dizi.length; i++) {
7      takas(dizi, k, i);
8      permutasyon(dizi, k + 1);
9      takas(dizi, k, i);
10   }
11 }
```



k = 2 → 3: yazdır

k = 1 düzeyi i = 1 ile başlar. k == 3: üçüncü permütasyon **[2, 1, 3]** yazdırılır.

Adım 5 / 10

ÇIKTI

```
[1, 2, 3]
[1, 3, 2]
[2, 1, 3]
```

DEĞİŞKENLER

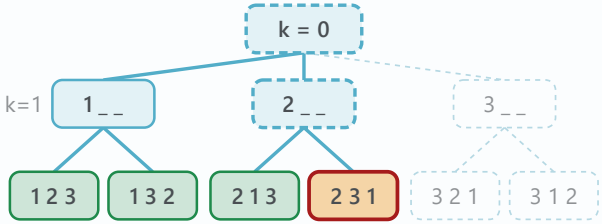
`k = 3` `i = –` `dizi = [2, 1, 3]`

permutasyon([1, 2, 3], 0): Adım Adım

```
1  void permutasyon(int[] dizi, int k) {
▶    if (k == dizi.length) {
▶      System.out.println(Arrays.toString(dizi));
4      return;
5    }
6    for (int i = k; i < dizi.length; i++) {
7      takas(dizi, k, i);
8      permutasyon(dizi, k + 1);
9      takas(dizi, k, i);
10   }
11 }
```

ÇIKTI

```
[1, 2, 3]
[1, 3, 2]
[2, 1, 3]
[2, 3, 1]
```



k = 2 → 3: yazdır

k = 1 düzeyinde önceki takas geri alındı; i = 2: `takas(dizi, 1, 2)`, ardından k = 2'de tek seçenek kalır. k == 3: dördüncü permütasyon **[2, 3, 1]** yazdırılır.

Adım 6 / 10

DEĞİŞKENLER

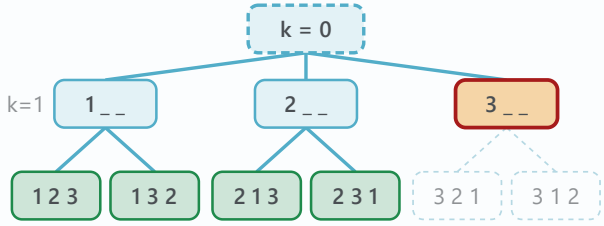
`k = 3` `i = –` `dizi = [2, 3, 1]`

permutasyon([1, 2, 3], 0): Adım Adım

```
1  void permutasyon(int[] dizi, int k) {
2    if (k == dizi.length) {
3      System.out.println(Arrays.toString(dizi));
4      return;
5    }
▶    for (int i = k; i < dizi.length; i++) {
▶      takas(dizi, k, i);
8      permutasyon(dizi, k + 1);
▶      takas(dizi, k, i);
10   }
11 }
```

ÇIKTI

```
[1, 2, 3]
[1, 3, 2]
[2, 1, 3]
[2, 3, 1]
```



k = 2 → 3: yazdır

Geri dönüşte `takas(dizi, 0, 1)` dizinin eski hâlini ([1, 2, 3]) geri getirir. i = 2: `takas(dizi, 0, 2)` → [3, 2, 1]; ilk konumda artık 3 var.

Adım 7 / 10

DEĞİŞKENLER

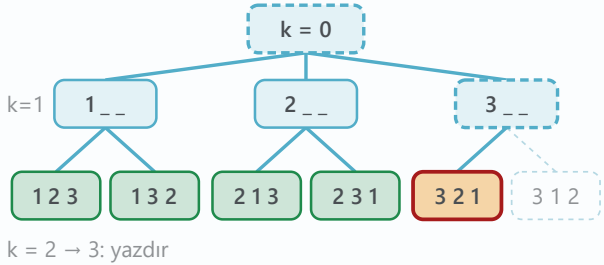
`k = 0` `i = 2` `dizi = [3, 2, 1]`

permutasyon([1, 2, 3], 0): Adım Adım

```
1  void permutasyon(int[] dizi, int k) {
▶    if (k == dizi.length) {
▶      System.out.println(Arrays.toString(dizi));
4      return;
5    }
6    for (int i = k; i < dizi.length; i++) {
7      takas(dizi, k, i);
8      permutasyon(dizi, k + 1);
9      takas(dizi, k, i);
10   }
11 }
```

ÇIKTI

```
[1, 2, 3]
[1, 3, 2]
[2, 1, 3]
[2, 3, 1]
[3, 2, 1]
```



k = 1 düzeyi i = 1 ile başlar. k == 3: beşinci permütasyon **[3, 2, 1]** yazdırılır.

Adım 8 / 10

DEĞİŞKENLER

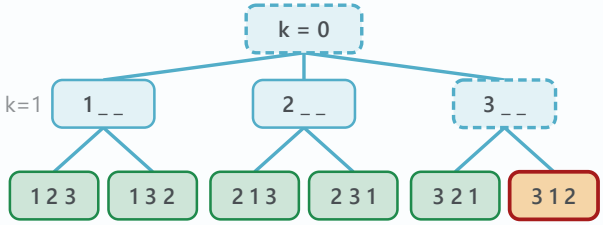
`k = 3` `i = –` `dizi = [3, 2, 1]`

permutasyon([1, 2, 3], 0): Adım Adım

```
1  void permutasyon(int[] dizi, int k) {
▶    if (k == dizi.length) {
▶      System.out.println(Arrays.toString(dizi));
4      return;
5    }
6    for (int i = k; i < dizi.length; i++) {
7      takas(dizi, k, i);
8      permutasyon(dizi, k + 1);
9      takas(dizi, k, i);
10   }
11 }
```

ÇIKTI

```
[1, 2, 3]
[1, 3, 2]
[2, 1, 3]
[2, 3, 1]
[3, 2, 1]
[3, 1, 2]
```



k = 2 → 3: yazdır

k = 1 düzeyinde önceki takas geri alındı; i = 2: `takas(dizi, 1, 2)`, ardından k = 2'de tek seçenek kalır. k == 3: altıncı permütasyon **[3, 1, 2]** yazdırılır.

Adım 9 / 10

DEĞİŞKENLER

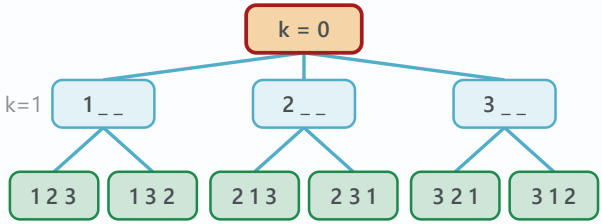
k = 3 | i = – | dizi = [3, 1, 2]

permutasyon([1, 2, 3], 0): Adım Adım

```
1  void permutasyon(int[] dizi, int k) {
2    if (k == dizi.length) {
3      System.out.println(Arrays.toString(dizi));
4      return;
5    }
6    for (int i = k; i < dizi.length; i++) {
7      takas(dizi, k, i);
8      permutasyon(dizi, k + 1);
▶      takas(dizi, k, i);
10   }
11 }
```

ÇIKTI

```
[1, 2, 3]
[1, 3, 2]
[2, 1, 3]
[2, 3, 1]
[3, 2, 1]
[3, 1, 2]
```



k = 2 → 3: yazdır

Son takas da geri alınır: dizi yine [1, 2, 3]. Döngü biter. Ağaçta 3 · 2 · 1 = **3! = 6** yaprak vardır.

Adım 10 / 10

DEĞİŞKENLER

`k = 0` `i = 2` `dizi = [1, 2, 3]`

Üstel Zaman: Fibonacci Serisi

```
long fib(int n) {           // saf özyineleme
  if (n <= 1) return n;
  return fib(n - 1) + fib(n - 2);
}

long[] bellek = new long[N + 1]; // 0: yok
long fibBellek(int n) {     // ezberleme
  if (n <= 1) return n;
  if (bellek[n] == 0)
    bellek[n] = fibBellek(n - 1) + fibBellek(n - 2);
  return bellek[n];
}
```

$F(0) = 0$, $F(1) = 1$, $F(n) = F(n-1) + F(n-2)$.

n	fib çağrısı	fibBellek
5	15	9
10	177	19
20	21.891	39
30	2.692.537	59

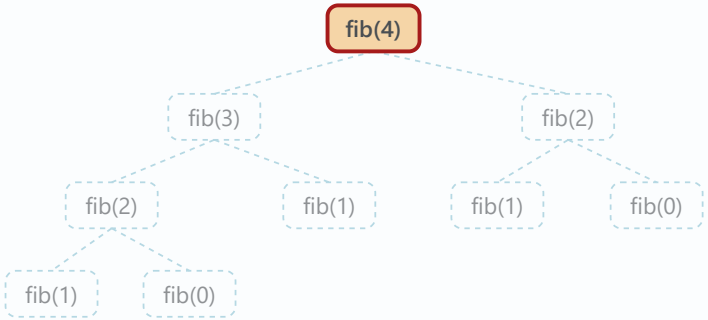
fib: her çağrı iki çağrı doğurur → **$O(2^n)$** (kesin olarak $\Theta(1{,}618^n)$).
fibBellek: her $F(k)$ bir kez hesaplanır, $2n - 1$ çağrı → **$O(n)$**.

fib(4): Çağrıları Adım Adım İzle

```
1 long fib(int n) {
2   if (n <= 1) return n;
▶   return fib(n - 1) + fib(n - 2);
4 }
```

ÇAĞRI YIĞINI

`fib(4)`



`fib(4)` : 4 > 1, taban durum değil. Önce `fib(3)` hesaplanacak; fib(4) yığında bekler.

Adım 1 / 13

DEĞİŞKENLER

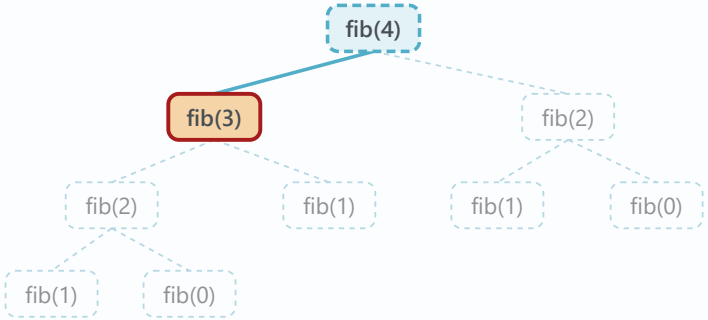
`n = 4` `sonuç = –` `çağrı = 1`

fib(4): Çağrıları Adım Adım İzle

```
1 long fib(int n) {
2   if (n <= 1) return n;
▶   return fib(n - 1) + fib(n - 2);
4 }
```

ÇAĞRI YIĞINI

- **fib(3)**
- fib(4)



`fib(3)` çağrıldı: 3 > 1 → önce `fib(2)`.

Adım 2 / 13

DEĞİŞKENLER

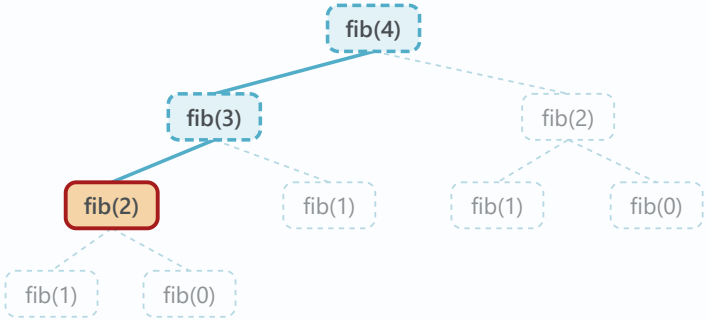
`n = 3` `sonuç = –` `çağrı = 2`

fib(4): Çağrıları Adım Adım İzle

```
1 long fib(int n) {
2   if (n <= 1) return n;
▶   return fib(n - 1) + fib(n - 2);
4 }
```

ÇAĞRI YIĞINI

- **fib(2)**
- fib(3)
- fib(4)



`fib(2)` çağrıldı: 2 > 1 → önce `fib(1)` .

Adım 3 / 13

DEĞİŞKENLER

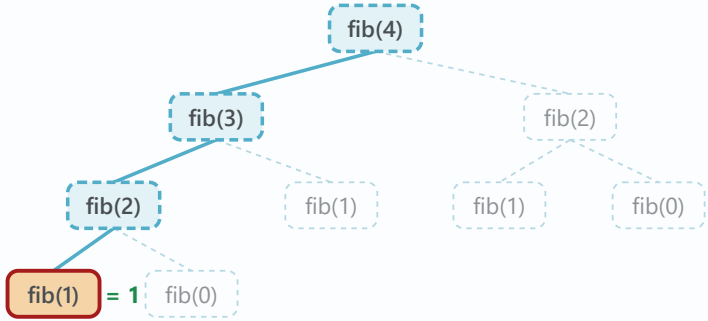
`n = 2` `sonuç = –` `çağrı = 3`

fib(4): Çağrıları Adım Adım İzle

```
1 long fib(int n) {
▶   if (n <= 1) return n;
3   return fib(n - 1) + fib(n - 2);
4 }
```

ÇAĞRI YIĞINI

- **fib(1)**
- fib(2)
- fib(3)
- fib(4)



`fib(1)` : n ≤ 1 → taban durum, **1** döner.

Adım 4 / 13

DEĞİŞKENLER

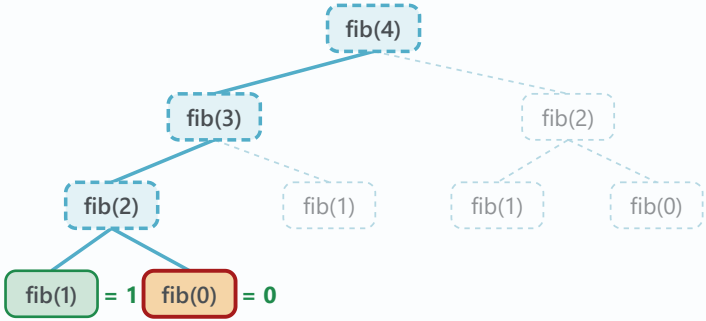
`n = 1` `sonuç = 1` `çağrı = 4`

fib(4): Çağrıları Adım Adım İzle

```
1 long fib(int n) {
▶   if (n <= 1) return n;
3   return fib(n - 1) + fib(n - 2);
4 }
```

ÇAĞRI YIĞINI

- **fib(0)**
- fib(2)
- fib(3)
- fib(4)



`fib(0)` : n ≤ 1 → taban durum, **0** döner.

Adım 5 / 13

DEĞİŞKENLER

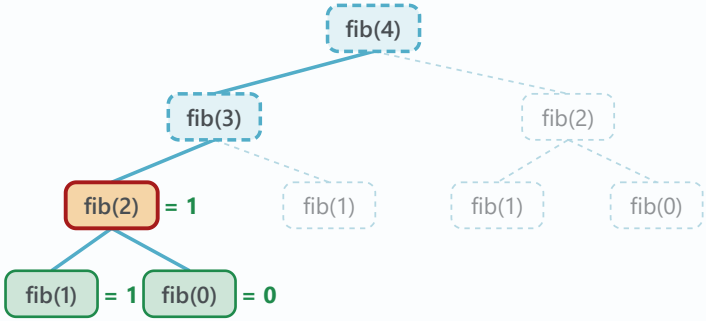
`n = 0` `sonuç = 0` `çağrı = 5`

fib(4): Çağrıları Adım Adım İzle

```
1 long fib(int n) {
2   if (n <= 1) return n;
▶   return fib(n - 1) + fib(n - 2);
4 }
```

ÇAĞRI YIĞINI

- **fib(2)**
- fib(3)
- fib(4)



İki alt çağrı bitti: fib(2) = 1 + 0 = **1** döner.

Adım 6 / 13

DEĞİŞKENLER

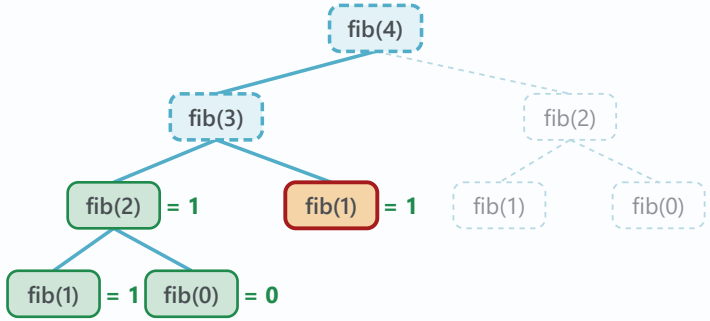
`n = 2` `sonuç = 1` `çağrı = 5`

fib(4): Çağrıları Adım Adım İzle

```
1 long fib(int n) {
▶   if (n <= 1) return n;
3   return fib(n - 1) + fib(n - 2);
4 }
```

ÇAĞRI YIĞINI

- `fib(1)`
- `fib(3)`
- `fib(4)`



`fib(1)` : n ≤ 1 → taban durum, **1** döner.

Adım 7 / 13

DEĞİŞKENLER

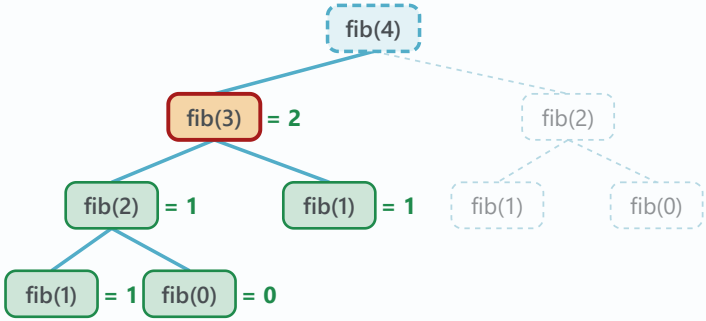
`n = 1` `sonuç = 1` `çağrı = 6`

fib(4): Çağrıları Adım Adım İzle

```
1 long fib(int n) {
2   if (n <= 1) return n;
▶   return fib(n - 1) + fib(n - 2);
4 }
```

ÇAĞRI YIĞINI

- **fib(3)**
- fib(4)



İki alt çağrı bitti: fib(3) = 1 + 1 = **2** döner.

Adım 8 / 13

DEĞİŞKENLER

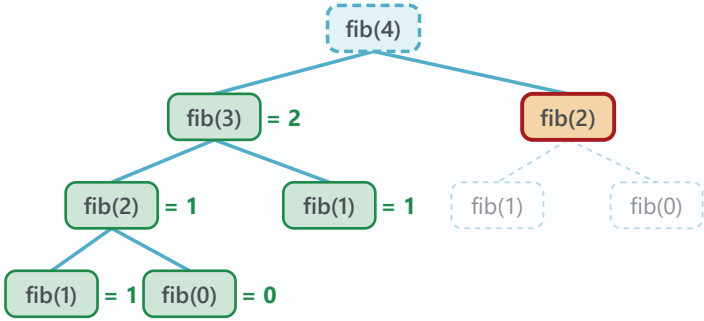
`n = 3` `sonuç = 2` `çağrı = 6`

fib(4): Çağrıları Adım Adım İzle

```
1 long fib(int n) {
2   if (n <= 1) return n;
▶   return fib(n - 1) + fib(n - 2);
4 }
```

ÇAĞRI YIĞINI

- **fib(2)**
- fib(4)



fib(4)'ün ikinci çağrısı `fib(2)`. Bu değer az önce hesaplanmıştı ama saf özyineleme hatırlamaz: **yeniden** hesaplanır.

DEĞİŞKENLER

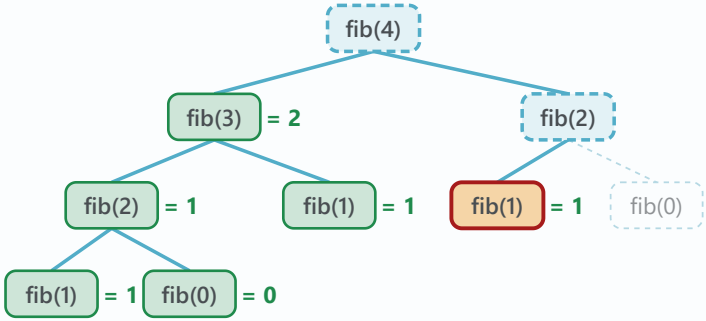
`n = 2` `sonuç = –` `çağrı = 7`

fib(4): Çağrıları Adım Adım İzle

```
1 long fib(int n) {
▶   if (n <= 1) return n;
3   return fib(n - 1) + fib(n - 2);
4 }
```

ÇAĞRI YIĞINI

- `fib(1)`
- `fib(2)`
- `fib(4)`



`fib(1)` : n ≤ 1 → taban durum, **1** döner.

Adım 10 / 13

DEĞİŞKENLER

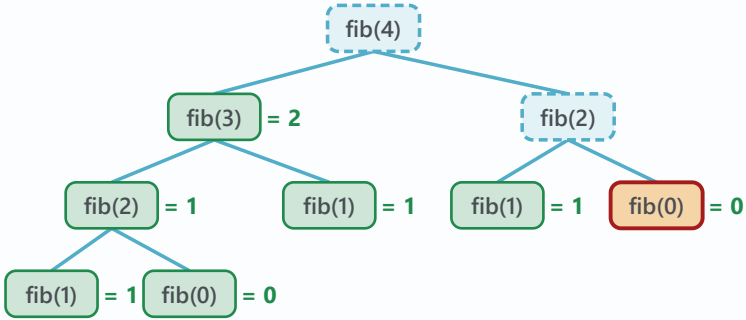
`n = 1` `sonuç = 1` `çağrı = 8`

fib(4): Çağrıları Adım Adım İzle

```
1 long fib(int n) {
▶   if (n <= 1) return n;
3   return fib(n - 1) + fib(n - 2);
4 }
```

ÇAĞRI YIĞINI

- **fib(0)**
- fib(2)
- fib(4)



`fib(0)` : n ≤ 1 → taban durum, **0** döner.

Adım 11 / 13

DEĞİŞKENLER

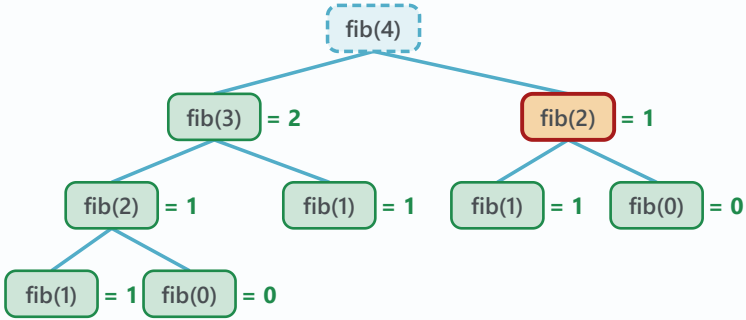
`n = 0` `sonuç = 0` `çağrı = 9`

fib(4): Çağrıları Adım Adım İzle

```
1 long fib(int n) {
2   if (n <= 1) return n;
▶   return fib(n - 1) + fib(n - 2);
4 }
```

ÇAĞRI YIĞINI

- **fib(2)**
- fib(4)



İki alt çağrı bitti: fib(2) = 1 + 0 = **1** döner.

Adım 12 / 13

DEĞİŞKENLER

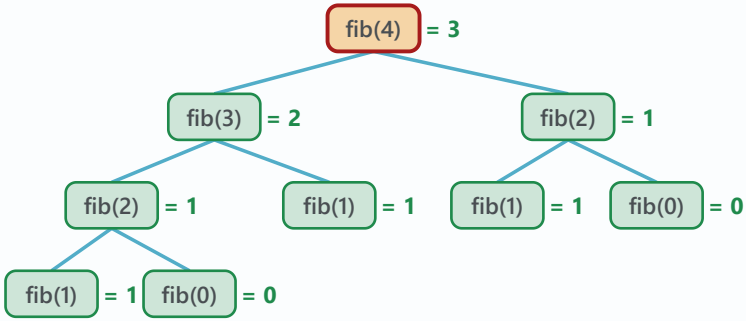
`n = 2` `sonuç = 1` `çağrı = 9`

fib(4): Çağrıları Adım Adım İzle

```
1 long fib(int n) {
2   if (n <= 1) return n;
▶   return fib(n - 1) + fib(n - 2);
4 }
```

ÇAĞRI YIĞINI

`fib(4)`



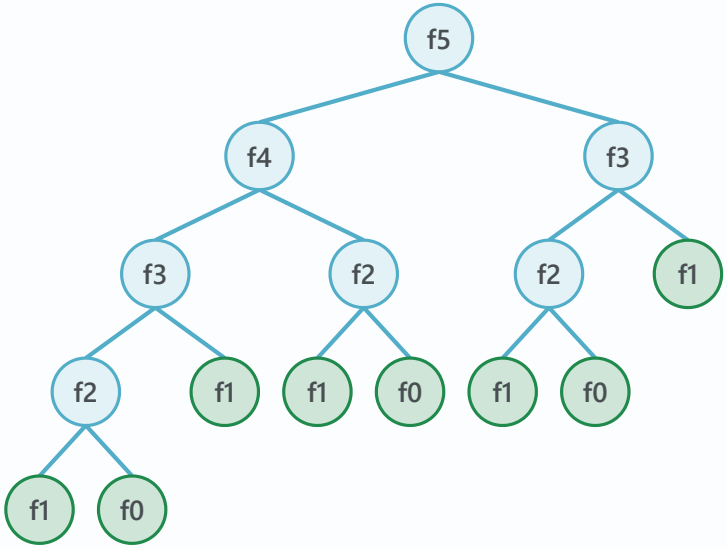
fib(4) = fib(3) + fib(2) = 2 + 1 = **3**. Toplam **9 çağrı**; fib(2) iki kez, fib(1) üç kez hesaplandı.

Adım 13 / 13

DEĞİŞKENLER

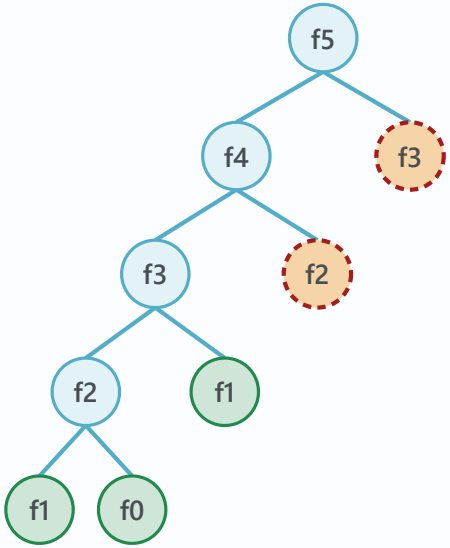
`n = 4` `sonuç = 3` `çağrı = 9`

Fibonacci: Çağrı Ağacı fib(5)



Saf özyineleme: **15 çağrı**. f3 iki kez, f2 üç kez, f1 beş kez hesaplanır; ağaç her düzeyde yaklaşık ikiye katlanır → üstel büyüme.

Fibonacci: Çağrı Ağacı fib(5)



Ezberlemeli: **9 çağrı**. Turuncu kesikli düğümler (f2, f3) bellek dizisinden **hazır** döner, alt ağaçları hiç açılmaz → O(n).

KARMAŞIKLIK ÖRNEKLERİ

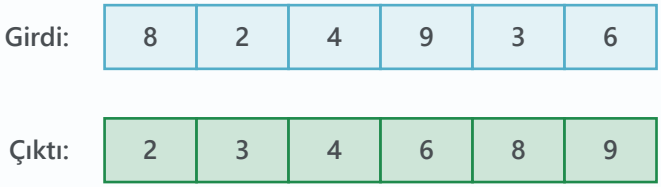
Karmaşıklık Sınıfları: Örneklerle

Sınıf	Adı	Örnek	n iki katına çıkınca süre
$O(1)$	sabit	ilk öğeyi yazdırma, dizi erişimi	değişmez
$O(\log n)$	logaritmik	ikili arama, hızlı üs alma, Öklid	+1 adım
$O(\sqrt{n})$	karekök	asal sayı kontrolü (deneme bölmesi)	≈ 1,41 katı
$O(n)$	doğrusal	doğrusal arama, en büyük öğe, birleştirme	2 katı
$O(n \log n)$	log-doğrusal	birleştirme sıralaması, hızlı sıralama (ortalama)	2 katından biraz fazla
$O(n^2)$	karesel	kabarcık, eklemeli sıralama	4 katı
$O(n^3)$	kübik	klasik matris çarpımı	8 katı
$O(2^n)$	üstel	saf özyinelemeli Fibonacci, alt kümeler	karesi alınır
$O(n!)$	faktöriyel	tüm permütasyonlar	çok daha hızlı patlar

Yukarıdan aşağıya büyüme hızı artar: $1 < \log n < \sqrt{n} < n < n \log n < n^2 < n^3 < 2^n < n!$. Polinom sınıflar ($n^k$) genelde "verimli", üstel olanlar "verimsiz" sayılır.

Sıralama Problemi

- **Girdi:** n sayıdan oluşan dizi $\langle a_1, a_2, \ldots, a_n \rangle$.
- **Çıktı:** aynı sayıların bir permütasyonu $\langle a'_1, a'_2, \ldots, a'_n \rangle$ öyle ki $a'_1 \leq a'_2 \leq \ldots \leq a'_n$.



Neden sıralama?

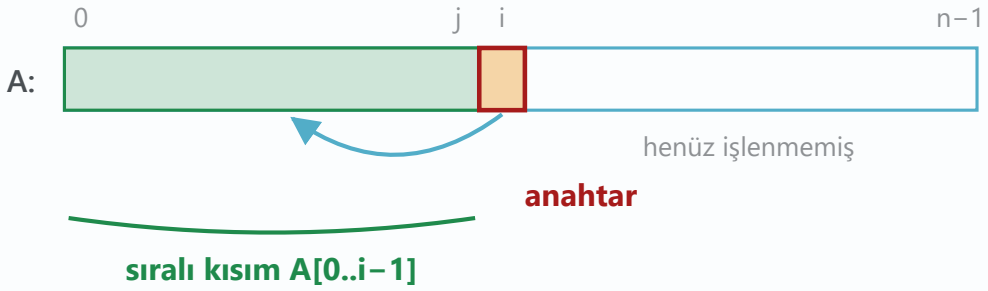
Arama (ikili arama sıralı dizi ister), tekrar bulma, sıralı rapor, birleştirme gibi pek çok işlemin ön adımıdır.

Örnek girdi

8 2 4 9 3 6

Eklemeli Sıralama (Insertion Sort)

- Diziyi sıralamak için **iç içe iki döngü** kullanır.
- **Dış döngü** dizinin her öğesini bir kez geçer: A[i] `anahtar` değişkenine alınır.
- **İç döngü** anahtardan büyük öğeleri birer sağa kaydırır ve anahtarı uygun konuma yerleştirir.
- İskambil kâğıtlarını elde sıralamak gibi: her yeni kâğıt, sıralı eldeki yerine sokulur.



Değişmez (invariant): her dış adımın başında A[0..i−1] sıralıdır. Zaman karmaşıklığı (en kötü) $O(n^2)$.

Eklemeli Sıralama: Kod

```
public void Sirala(int[] A) {
  for (int i = 1; i < A.length; ++i) {
    int anahtar = A[i];
    int j = i - 1;
    // A[0..i-1] alt dizisi sıralı.
    // anahtar uygun konuma yerleştirilir.
    while (j >= 0 && A[j] > anahtar) {
      A[j + 1] = A[j];
      j = j - 1;
    }
    A[j + 1] = anahtar;
  }
}
```

Dış döngü: n − 1 adım

i = 1, 2, …, n−1. Her adımda bir öğe sıralı kısma eklenir.

İç döngü: en fazla i adım

Anahtar, sıralı kısımdaki tüm öğelerden küçükse (dizi tersten sıralıysa) i öğe kaydırılır.

Toplam (en kötü)

1 + 2 + … + (n−1) = n(n−1)/2 → **$O(n^2)$**.
Ek bellek O(1): yerinde (in-place) sıralar.

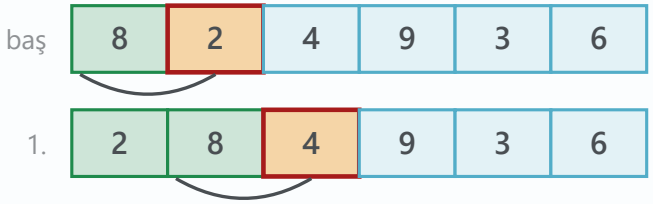
EKLEMELİ SIRALAMA

Örnek: 8 2 4 9 3 6



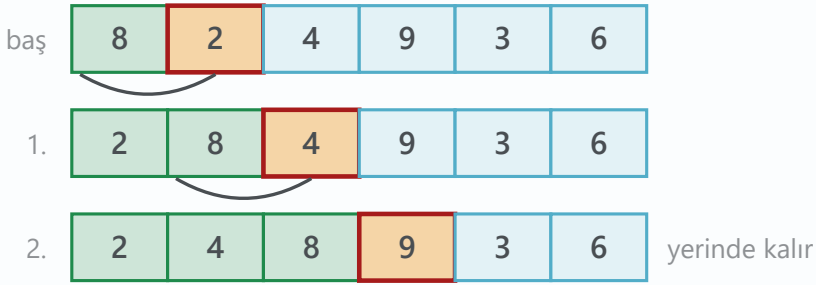
Başlangıç: 8 tek başına sıralıdır (yeşil). Anahtar **2**: 8'den küçük → 8 sağa kayar, 2 başa girer.

Örnek: 8 2 4 9 3 6



1. ekleme sonrası: 2 8 4 9 3 6. Sıradaki anahtar **4**: kendinden büyük 1 öğe sağa kayar, 1. konuma girer.

Örnek: 8 2 4 9 3 6



2. ekleme sonrası: 2 4 8 9 3 6. Sıradaki anahtar **9**: solundaki 8'ten büyük, yerinde kalır.

Örnek: 8 2 4 9 3 6



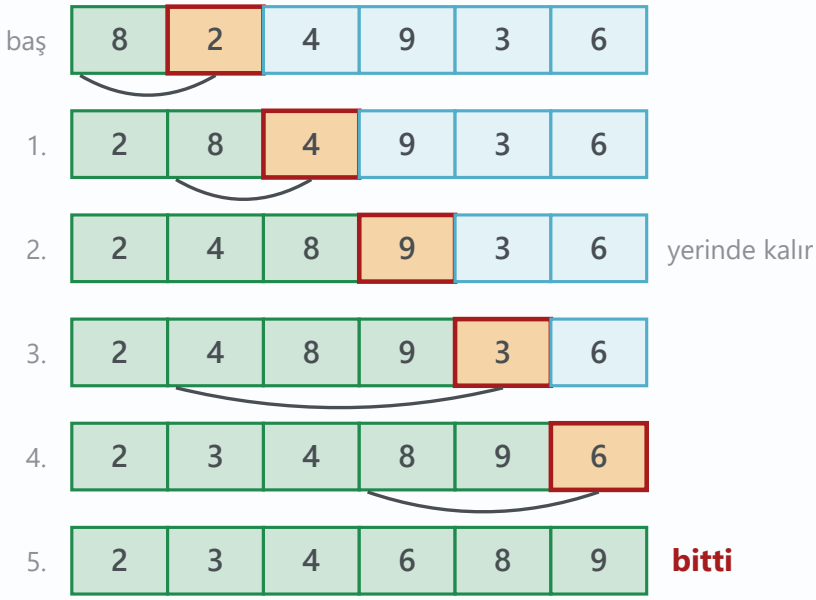
3. ekleme sonrası: 2 4 8 9 3 6. Sıradaki anahtar **3**: kendinden büyük 3 öğe sağa kayar, 1. konuma girer.

Örnek: 8 2 4 9 3 6



4. ekleme sonrası: 2 3 4 8 9 6. Sıradaki anahtar **6**: kendinden büyük 2 öğe sağa kayar, 3. konuma girer.

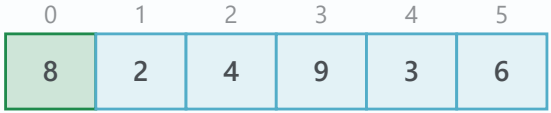
Örnek: 8 2 4 9 3 6



Son öğe da yerine girdi: **2 3 4 6 8 9** — sıralama bitti (n − 1 = 5 dış adım).

Sirala(A): Kodu Adım Adım İzle

```
▶ public void Sirala(int[] A) {
2   for (int i = 1; i < A.length; ++i) {
3     int anahtar = A[i];
4     int j = i - 1;
5     // A[0..i-1] sıralı
6     // anahtar yerine girer
7     while (j >= 0 && A[j] > anahtar) {
8       A[j + 1] = A[j];
9       j = j - 1;
10    }
11    A[j + 1] = anahtar;
12  }
13 }
```



Dizi: 8 2 4 9 3 6. A[0] = 8 tek başına sıralı kabul edilir; dış döngü i = 1'den başlar.

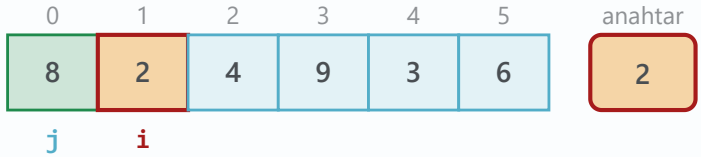
Adım 1 / 19

DEĞİŞKENLER

`i = –` `anahtar = –` `j = –`

Sirala(A): Kodu Adım Adım İzle

```
1  public void Sirala(int[] A) {
2    for (int i = 1; i < A.length; ++i) {
▶      int anahtar = A[i];
▶      int j = i - 1;
5      // A[0..i-1] sıralı
6      // anahtar yerine girer
7      while (j >= 0 && A[j] > anahtar) {
8        A[j + 1] = A[j];
9        j = j - 1;
10     }
11     A[j + 1] = anahtar;
12   }
13 }
```



i = 1: `anahtar = A[1] = 2` , `j = 0` . Sıralı kısım: 8.

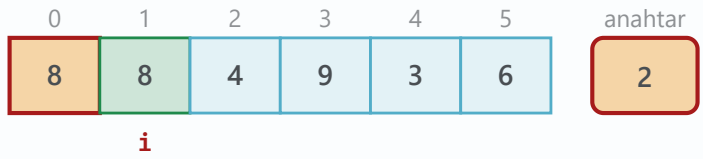
Adım 2 / 19

DEĞİŞKENLER

`i = 1` `anahtar = 2` `j = 0`

Sirala(A): Kodu Adım Adım İzle

```
1  public void Sirala(int[] A) {
2    for (int i = 1; i < A.length; ++i) {
3      int anahtar = A[i];
4      int j = i - 1;
5      // A[0..i-1] sıralı
6      // anahtar yerine girer
7      while (j >= 0 && A[j] > anahtar) {
▶        A[j + 1] = A[j];
▶        j = j - 1;
10     }
11     A[j + 1] = anahtar;
12   }
13 }
```



A[0] = 8 > 2 → `A[1] = A[0]` (sağa kaydır), `j = -1`.

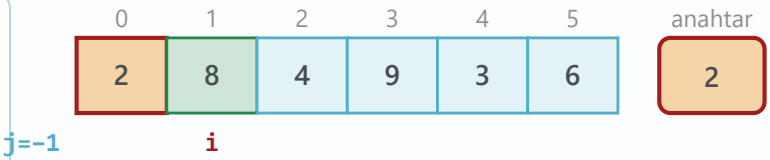
Adım 3 / 19

DEĞİŞKENLER

`i = 1` `anahtar = 2` `j = -1`

Sirala(A): Kodu Adım Adım İzle

```
1  public void Sirala(int[] A) {
2    for (int i = 1; i < A.length; ++i) {
3      int anahtar = A[i];
4      int j = i - 1;
5      // A[0..i-1] sıralı
6      // anahtar yerine girer
▶      while (j >= 0 && A[j] > anahtar) {
8        A[j + 1] = A[j];
9        j = j - 1;
10     }
▶      A[j + 1] = anahtar;
12   }
13 }
```



Koşul yanlış: j = −1 (başa gelindi). `A[0] = anahtar = 2`. A[0..1] sıralı: 2 8.

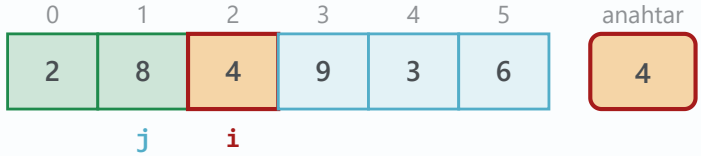
Adım 4 / 19

DEĞİŞKENLER

`i = 1` `anahtar = 2` `j = -1`

Sirala(A): Kodu Adım Adım İzle

```
1  public void Sirala(int[] A) {
2    for (int i = 1; i < A.length; ++i) {
▶      int anahtar = A[i];
▶      int j = i - 1;
5      // A[0..i-1] sıralı
6      // anahtar yerine girer
7      while (j >= 0 && A[j] > anahtar) {
8        A[j + 1] = A[j];
9        j = j - 1;
10     }
11     A[j + 1] = anahtar;
12   }
13 }
```



i = 2: `anahtar = A[2] = 4` , `j = 1` . Sıralı kısım: 2 8.

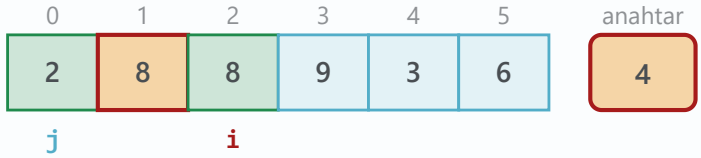
Adım 5 / 19

DEĞİŞKENLER

`i = 2` `anahtar = 4` `j = 1`

Sirala(A): Kodu Adım Adım İzle

```
1  public void Sirala(int[] A) {
2    for (int i = 1; i < A.length; ++i) {
3      int anahtar = A[i];
4      int j = i - 1;
5      // A[0..i-1] sıralı
6      // anahtar yerine girer
7      while (j >= 0 && A[j] > anahtar) {
▶        A[j + 1] = A[j];
▶        j = j - 1;
10     }
11     A[j + 1] = anahtar;
12   }
13 }
```



A[1] = 8 > 4 → `A[2] = A[1]` (sağa kaydır), `j = 0`.

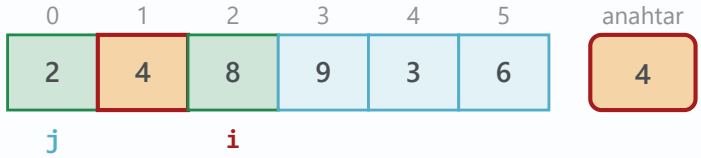
Adım 6 / 19

DEĞİŞKENLER

`i = 2` `anahtar = 4` `j = 0`

Sirala(A): Kodu Adım Adım İzle

```
1  public void Sirala(int[] A) {
2    for (int i = 1; i < A.length; ++i) {
3      int anahtar = A[i];
4      int j = i - 1;
5      // A[0..i-1] sıralı
6      // anahtar yerine girer
▶      while (j >= 0 && A[j] > anahtar) {
8        A[j + 1] = A[j];
9        j = j - 1;
10     }
▶      A[j + 1] = anahtar;
12   }
13 }
```



Koşul yanlış: A[0] = 2 ≤ 4. `A[1] = anahtar = 4`. A[0..2] sıralı: 2 4 8.

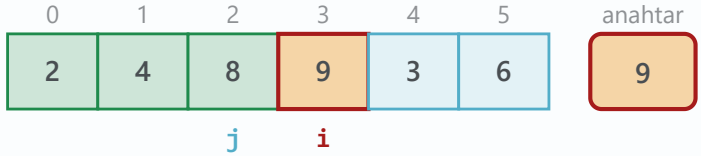
Adım 7 / 19

DEĞİŞKENLER

`i = 2` `anahtar = 4` `j = 0`

Sirala(A): Kodu Adım Adım İzle

```
1  public void Sirala(int[] A) {
2    for (int i = 1; i < A.length; ++i) {
▶      int anahtar = A[i];
▶      int j = i - 1;
5      // A[0..i-1] sıralı
6      // anahtar yerine girer
7      while (j >= 0 && A[j] > anahtar) {
8        A[j + 1] = A[j];
9        j = j - 1;
10     }
11     A[j + 1] = anahtar;
12   }
13 }
```



i = 3: `anahtar = A[3] = 9` , `j = 2` . Sıralı kısım: 2 4 8.

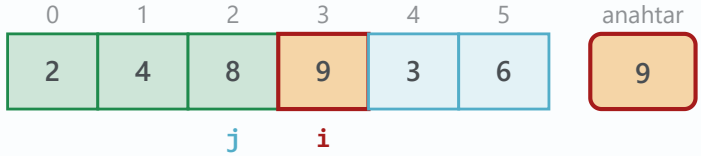
Adım 8 / 19

DEĞİŞKENLER

`i = 3` `anahtar = 9` `j = 2`

Sirala(A): Kodu Adım Adım İzle

```
1  public void Sirala(int[] A) {
2    for (int i = 1; i < A.length; ++i) {
3      int anahtar = A[i];
4      int j = i - 1;
5      // A[0..i-1] sıralı
6      // anahtar yerine girer
▶      while (j >= 0 && A[j] > anahtar) {
8        A[j + 1] = A[j];
9        j = j - 1;
10     }
▶      A[j + 1] = anahtar;
12   }
13 }
```



Koşul yanlış: A[2] = 8 ≤ 9. `A[3] = anahtar = 9`. A[0..3] sıralı: 2 4 8 9.

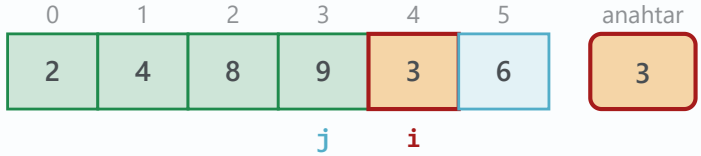
Adım 9 / 19

DEĞİŞKENLER

`i = 3` `anahtar = 9` `j = 2`

Sirala(A): Kodu Adım Adım İzle

```
1  public void Sirala(int[] A) {
2    for (int i = 1; i < A.length; ++i) {
▶      int anahtar = A[i];
▶      int j = i - 1;
5      // A[0..i-1] sıralı
6      // anahtar yerine girer
7      while (j >= 0 && A[j] > anahtar) {
8        A[j + 1] = A[j];
9        j = j - 1;
10     }
11     A[j + 1] = anahtar;
12   }
13 }
```



i = 4: `anahtar = A[4] = 3` , `j = 3` . Sıralı kısım: 2 4 8 9.

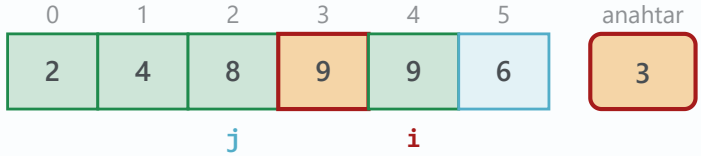
Adım 10 / 19

DEĞİŞKENLER

`i = 4` `anahtar = 3` `j = 3`

Sirala(A): Kodu Adım Adım İzle

```
1  public void Sirala(int[] A) {
2    for (int i = 1; i < A.length; ++i) {
3      int anahtar = A[i];
4      int j = i - 1;
5      // A[0..i-1] sıralı
6      // anahtar yerine girer
7      while (j >= 0 && A[j] > anahtar) {
▶        A[j + 1] = A[j];
▶        j = j - 1;
10     }
11     A[j + 1] = anahtar;
12   }
13 }
```



A[3] = 9 > 3 → `A[4] = A[3]` (sağa kaydır), `j = 2`.

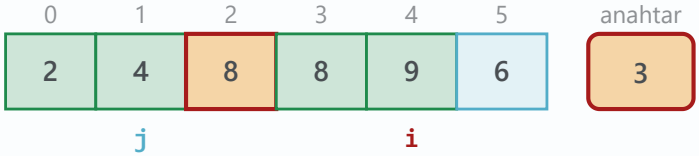
Adım 11 / 19

DEĞİŞKENLER

`i = 4` `anahtar = 3` `j = 2`

Sirala(A): Kodu Adım Adım İzle

```
1  public void Sirala(int[] A) {
2    for (int i = 1; i < A.length; ++i) {
3      int anahtar = A[i];
4      int j = i - 1;
5      // A[0..i-1] sıralı
6      // anahtar yerine girer
7      while (j >= 0 && A[j] > anahtar) {
▶        A[j + 1] = A[j];
▶        j = j - 1;
10     }
11     A[j + 1] = anahtar;
12   }
13 }
```



A[2] = 8 > 3 → `A[3] = A[2]` (sağa kaydır), `j = 1`.

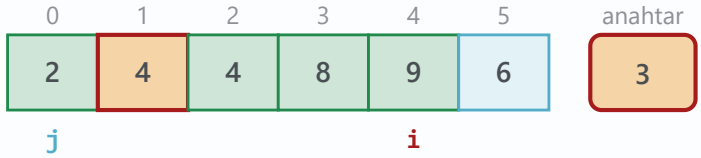
Adım 12 / 19

DEĞİŞKENLER

`i = 4` `anahtar = 3` `j = 1`

Sirala(A): Kodu Adım Adım İzle

```
1  public void Sirala(int[] A) {
2    for (int i = 1; i < A.length; ++i) {
3      int anahtar = A[i];
4      int j = i - 1;
5      // A[0..i-1] sıralı
6      // anahtar yerine girer
7      while (j >= 0 && A[j] > anahtar) {
▶        A[j + 1] = A[j];
▶        j = j - 1;
10     }
11     A[j + 1] = anahtar;
12   }
13 }
```



A[1] = 4 > 3 → `A[2] = A[1]` (sağa kaydır), `j = 0`.

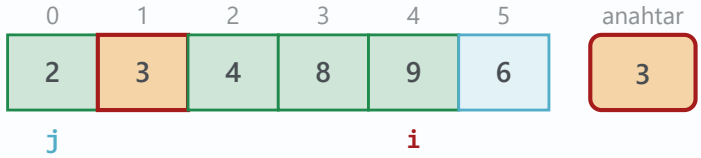
Adım 13 / 19

DEĞİŞKENLER

`i = 4` `anahtar = 3` `j = 0`

Sirala(A): Kodu Adım Adım İzle

```
1  public void Sirala(int[] A) {
2    for (int i = 1; i < A.length; ++i) {
3      int anahtar = A[i];
4      int j = i - 1;
5      // A[0..i-1] sıralı
6      // anahtar yerine girer
▶      while (j >= 0 && A[j] > anahtar) {
8        A[j + 1] = A[j];
9        j = j - 1;
10     }
▶      A[j + 1] = anahtar;
12   }
13 }
```



Koşul yanlış: A[0] = 2 ≤ 3. `A[1] = anahtar = 3`. A[0..4] sıralı: 2 3 4 8 9.

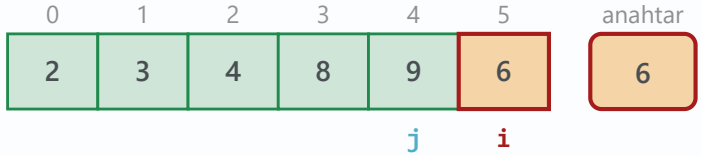
Adım 14 / 19

DEĞİŞKENLER

`i = 4` `anahtar = 3` `j = 0`

Sirala(A): Kodu Adım Adım İzle

```
1  public void Sirala(int[] A) {
2    for (int i = 1; i < A.length; ++i) {
▶      int anahtar = A[i];
▶      int j = i - 1;
5      // A[0..i-1] sıralı
6      // anahtar yerine girer
7      while (j >= 0 && A[j] > anahtar) {
8        A[j + 1] = A[j];
9        j = j - 1;
10     }
11     A[j + 1] = anahtar;
12   }
13 }
```



i = 5: `anahtar = A[5] = 6` , `j = 4` . Sıralı kısım: 2 3 4 8 9.

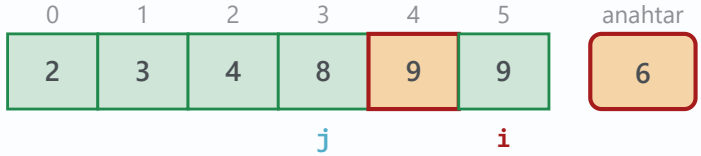
Adım 15 / 19

DEĞİŞKENLER

`i = 5` `anahtar = 6` `j = 4`

Sirala(A): Kodu Adım Adım İzle

```
1  public void Sirala(int[] A) {
2    for (int i = 1; i < A.length; ++i) {
3      int anahtar = A[i];
4      int j = i - 1;
5      // A[0..i-1] sıralı
6      // anahtar yerine girer
7      while (j >= 0 && A[j] > anahtar) {
▶        A[j + 1] = A[j];
▶        j = j - 1;
10     }
11     A[j + 1] = anahtar;
12   }
13 }
```



A[4] = 9 > 6 → `A[5] = A[4]` (sağa kaydır), `j = 3`.

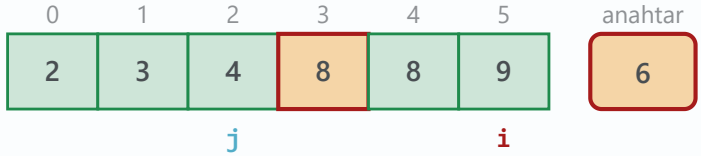
Adım 16 / 19

DEĞİŞKENLER

`i = 5` `anahtar = 6` `j = 3`

Sirala(A): Kodu Adım Adım İzle

```
1  public void Sirala(int[] A) {
2    for (int i = 1; i < A.length; ++i) {
3      int anahtar = A[i];
4      int j = i - 1;
5      // A[0..i-1] sıralı
6      // anahtar yerine girer
7      while (j >= 0 && A[j] > anahtar) {
▶        A[j + 1] = A[j];
▶        j = j - 1;
10     }
11     A[j + 1] = anahtar;
12   }
13 }
```



A[3] = 8 > 6 → `A[4] = A[3]` (sağa kaydır), `j = 2`.

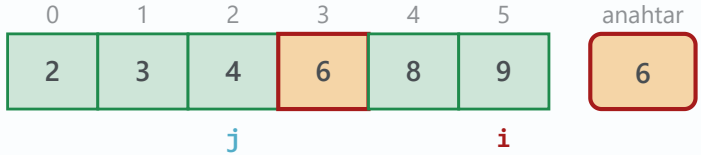
Adım 17 / 19

DEĞİŞKENLER

`i = 5` `anahtar = 6` `j = 2`

Sirala(A): Kodu Adım Adım İzle

```
1  public void Sirala(int[] A) {
2    for (int i = 1; i < A.length; ++i) {
3      int anahtar = A[i];
4      int j = i - 1;
5      // A[0..i-1] sıralı
6      // anahtar yerine girer
▶      while (j >= 0 && A[j] > anahtar) {
8        A[j + 1] = A[j];
9        j = j - 1;
10     }
▶      A[j + 1] = anahtar;
12   }
13 }
```



Koşul yanlış: A[2] = 4 ≤ 6. `A[3] = anahtar = 6`.
A[0..5] sıralı: 2 3 4 6 8 9.

Adım 18 / 19

DEĞİŞKENLER

`i = 5` `anahtar = 6` `j = 2`

Sirala(A): Kodu Adım Adım İzle

```
1  public void Sirala(int[] A) {
2    for (int i = 1; i < A.length; ++i) {
3      int anahtar = A[i];
4      int j = i - 1;
5      // A[0..i-1] sıralı
6      // anahtar yerine girer
7      while (j >= 0 && A[j] > anahtar) {
8        A[j + 1] = A[j];
9        j = j - 1;
10     }
11     A[j + 1] = anahtar;
12   }
13 }
```



i = 6 = A.length → döngü biter. Sonuç: **2 3 4 6 8 9**. Toplam 11 karşılaştırma, 7 kaydırma.

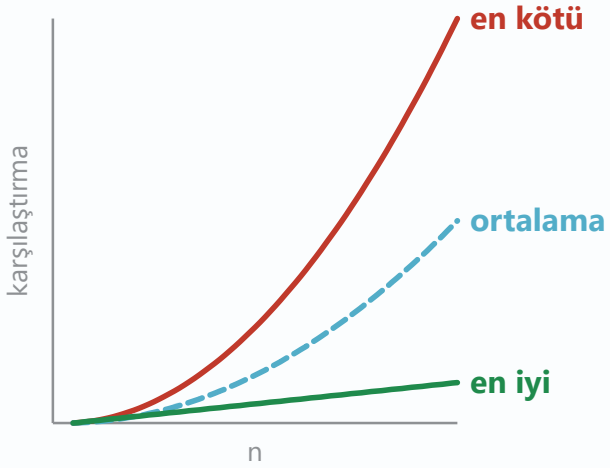
Adım 19 / 19

DEĞİŞKENLER

`i = 6` `anahtar = –` `j = –`

Çalışma Süresi ve Analiz Türleri

- **Girdi durumuna bağlı:** dizi zaten sıralıysa sıralamak kolaydır.
- **Girdi boyutuna bağlı:** az öğeli dizi kısa sürede sıralanır → T(n).
- **En kötü durum** (genellikle): n boyutlu herhangi bir girdi için **maksimum** süre. Herkes garanti ister.
- **Ortalama durum** (bazen): tüm girdiler üzerinden **beklenen** süre; girdi dağılımı hakkında varsayım gerekir.
- **En iyi durum** (yanıltıcı): özel, "hileli" bir girdide geçen süre; tek başına bir şey kanıtlamaz.



Eklemeli sıralama: **sıralı** girdi → n−1 karşılaştırma; **tersten sıralı** girdi → n(n−1)/2. Örnek dizimizde: 11 karşılaştırma.

Dikkat: Durum ve Sınır Farklı Kavramlardır

Durum (case)
Hangi **girdiye** bakıyoruz? En kötü, ortalama, en iyi girdi. Her durumun kendi süre fonksiyonu vardır.

Sınır (bound)
Bir fonksiyonu nasıl **özetliyoruz**? O: üst sınır, Ω: alt sınır, Θ: ikisi birden. Her duruma uygulanabilir.

Eklemeli sıralama	Sıkı sınır	Yorum
En iyi durum (sıralı girdi)	$\Theta(n)$	$n-1$ karşılaştırma
En kötü durum (ters sıralı girdi)	$\Theta(n^2)$	$n(n-1)/2$ karşılaştırma
Tüm girdiler için	$O(n^2)$ ve $\Omega(n)$	tek bir Θ yazılamaz

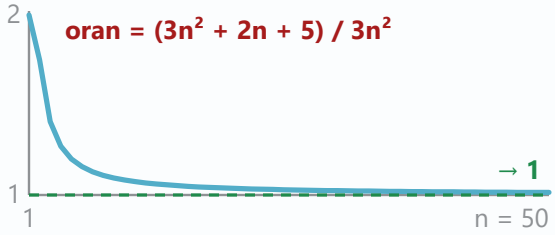
Dikkat: "En kötü O(n log n), en iyi Ω(n) ise çalışma süresi Θ(n log n)" çıkarımı yanlıştır. Θ ancak alt ve üst sınır **aynı** fonksiyon olduğunda yazılır; ör. birleştirme sıralaması her girdide Θ(n log n)'dir.

Makineden Bağımsız Çalışma Süresi

- Eklemeli sıralamanın en kötü durumu **$O(n^2)$**: girdi tersten sıralıysa gerçekleşir.
- Bilgisayarın hızına göre süre değişir; **asimptotik analiz** büyüme oranına odaklanır, makineye bağlı **sabit çarpanlar** göz ardı edilir.
- $n \to \infty$ giderken T(n)'nin büyüme oranına bakılır: yalnızca **en önemli** (üssü en büyük) terim değerlendirilir.
- **$T(n) = 3n^2 + 2n + 5 = O(n^2)$**

n	$3n^2+2n+5$	$3n^2$	oran
10	325	300	1,0833
100	30.205	30.000	1,0068
1.000	3.002.005	3.000.000	1,0007

n büyüdükçe düşük dereceli terimlerin payı sıfıra yaklaşır. 3 katsayısı ise makineye bağlıdır: hızlı bilgisayarda 1, yavaşta 10 olabilir → atılır.

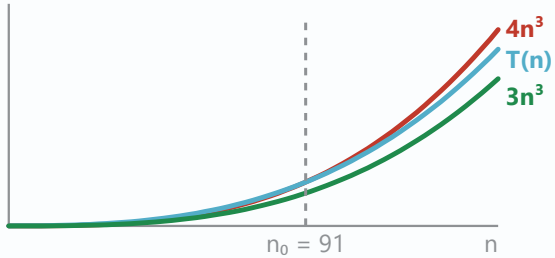


Θ Gösterimi: Tanım ve Uygulama

Θ(g(n)) = { f(n) : tüm n ≥ n_0 değerleri için **0 ≤ c_1 · g(n) ≤ f(n) ≤ c_2 · g(n)** şartını sağlayan pozitif c_1, c_2 ve n_0 sabitleri vardır }

Mühendislik kuralı:

1. $T(n) = 3n^3 + 90n^2 - 5n + 6046$
2. Düşük kuvvetli terimleri sil: $3n^3$
3. Katsayıları kaldır: **$T(n) = \Theta(n^3)$**



Tanımla doğrulama

$c_1 = 3$: $3n^3 \le T(n)$, çünkü $90n^2 - 5n + 6046 > 0$ (her $n \ge 1$).

$c_2 = 4$: $T(n) \le 4n^3 \Leftrightarrow 90n^2 - 5n + 6046 \le n^3$, bu da **$n \ge 91$** için doğrudur.

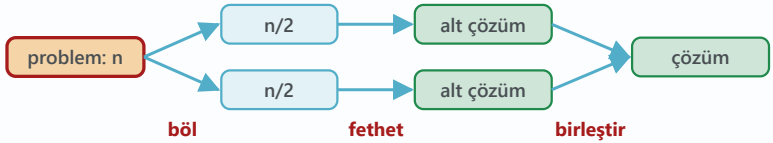
$\Rightarrow c_1 = 3$, $c_2 = 4$, $n_0 = 91$ ile $T(n) = \Theta(n^3)$.

Böl ve Fethet (Divide and Conquer)

1. Böl (Divide)
Problemi aynı türden, daha küçük alt problemlere ayır.

2. Fethet (Conquer)
Alt problemleri özyinelemeli çöz; yeterince küçükse doğrudan çöz.

3. Birleştir (Combine)
Alt çözümleri birleştirerek asıl problemin çözümünü elde et.



Algoritma	Böl	Fethet	Birleştir	T(n)
Birleştirme sıralaması	ortadan ikiye	2 yarıyı sırala	birleştirme: Θ(n)	2T(n/2) + Θ(n)
İkili arama	ortayı kontrol et	1 yarıda ara	önemsiz: Θ(1)	T(n/2) + Θ(1)
Üs alma a^n	n'yi ikiye böl	$a^{\lfloor n/2 \rfloor}$ hesapla	kare al: Θ(1)	T(n/2) + Θ(1)

Böyle özyinelemeli tanımlanan süre fonksiyonlarına **özyineleme bağıntısı** (recurrence) denir.

Birleştirme Sıralaması (Merge Sort)

Adım	Maliyet
BirleştirSırala A[1 . . n]	T(n)
1. n = 1 ise işlem tamam.	
2. liste1 = BirleştirSırala A[1 . . [n/2]]	
3. liste2 = BirleştirSırala A[[n/2]+1 . . n]	
4. İki sıralı listeyi birleştir.	

Birleştirme sıralaması (Merge Sort): n öğeli diziyi sıralamanın süresi **T(n)** olsun.

Birleştirme Sıralaması (Merge Sort)

Adım	Maliyet
BirleştirSırala A[1 . . n]	T(n)
1. n = 1 ise işlem tamam.	Θ(1)
2. liste1 = BirleştirSırala A[1 . . ⌈n/2⌉]	
3. liste2 = BirleştirSırala A[⌈n/2⌉+1 . . n]	
4. İki sıralı listeyi birleştir.	

n = 1 ise dizi zaten sıralıdır: sabit süre, **Θ(1)**.

Birleştirme Sıralaması (Merge Sort)

Adım	Maliyet
BirleştirSırala A[1 . . n]	T(n)
1. n = 1 ise işlem tamam.	Θ(1)
2. liste1 = BirleştirSırala A[1 . . ⌈n/2⌉]	T(n/2)
3. liste2 = BirleştirSırala A[⌈n/2⌉+1 . . n]	
4. İki sıralı listeyi birleştir.	

Sol yarıyı sıralamak, aynı problemin n/2 boyutlu hâlidir: **T(n/2)**.

Birleştirme Sıralaması (Merge Sort)

Adım	Maliyet
BirleştirSırala A[1 . . n]	T(n)
1. n = 1 ise işlem tamam.	Θ(1)
2. liste1 = BirleştirSırala A[1 . . ⌈n/2⌉]	T(n/2)
3. liste2 = BirleştirSırala A[⌈n/2⌉+1 . . n]	T(n/2)
4. İki sıralı listeyi birleştir.	

Sağ yarı da **T(n/2)** sürer.

Birleştirme Sıralaması (Merge Sort)

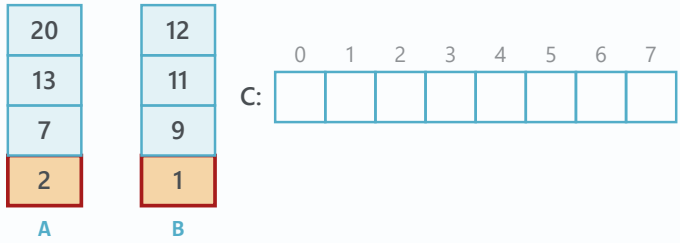
Adım	Maliyet
BirleştirSırala A[1 . . n]	T(n)
1. n = 1 ise işlem tamam.	Θ(1)
2. liste1 = BirleştirSırala A[1 . . ⌈n/2⌉]	T(n/2)
3. liste2 = BirleştirSırala A[⌈n/2⌉+1 . . n]	T(n/2)
4. İki sıralı listeyi birleştir.	Θ(n)

$$T(n) = \Theta(1),\ n = 1 \text{ ise;} \quad T(n) = 2T(n/2) + \Theta(n),\ n > 1 \text{ ise.}$$

İki sıralı listeyi birleştirmek her öğeyi bir kez yazar: **Θ(n)**. Böylece $T(n) = 2T(n/2) + \Theta(n)$.

Birleştirme İşlemi (Merge): Adım Adım

```
▶ int i = 0, j = 0, k = 0;
2 while (i < A.length && j < B.length) {
3   if (A[i] <= B[j])
4     C[k++] = A[i++];
5   else
6     C[k++] = B[j++];
7 }
8 while (i < A.length)
9   C[k++] = A[i++];
10 while (j < B.length)
11   C[k++] = B[j++];
```



İki sıralı liste: A = 2 7 13 20, B = 1 9 11 12 (en küçükler altta). Yalnızca iki listenin **en önündeki** (sarı) öğeler karşılaştırılır.

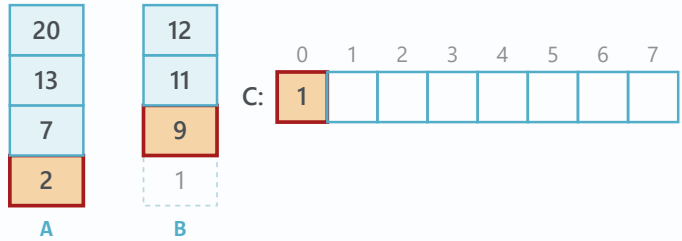
Adım 1 / 9

DEĞİŞKENLER

`i = 0` `j = 0` `k = 0`

Birleştirme İşlemi (Merge): Adım Adım

```
1  int i = 0, j = 0, k = 0;
2  while (i < A.length && j < B.length) {
3    if (A[i] <= B[j])
4      C[k++] = A[i++];
▶    else
▶      C[k++] = B[j++];
7  }
8  while (i < A.length)
9    C[k++] = A[i++];
10 while (j < B.length)
11   C[k++] = B[j++];
```



A[0] = 2 > B[0] = 1 → 1, C'ye yazılır; j ilerler.

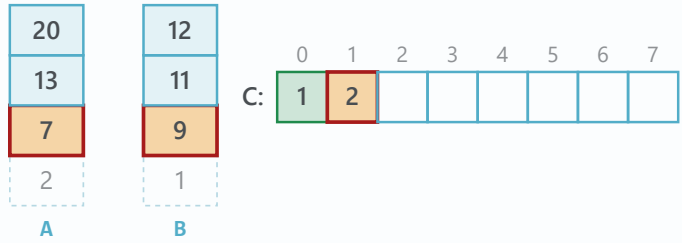
Adım 2 / 9

DEĞİŞKENLER

`i = 0` `j = 1` `k = 1`

Birleştirme İşlemi (Merge): Adım Adım

```
1  int i = 0, j = 0, k = 0;
2  while (i < A.length && j < B.length) {
▶    if (A[i] <= B[j])
▶      C[k++] = A[i++];
5    else
6      C[k++] = B[j++];
7  }
8  while (i < A.length)
9    C[k++] = A[i++];
10 while (j < B.length)
11   C[k++] = B[j++];
```



A[0] = 2 ≤ B[1] = 9 → 2, C'ye yazılır; i ilerler.

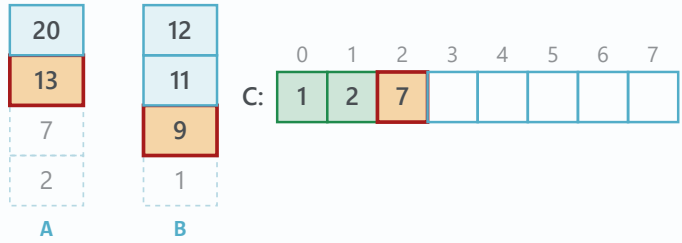
Adım 3 / 9

DEĞİŞKENLER

i = 1 j = 1 k = 2

Birleştirme İşlemi (Merge): Adım Adım

```
1  int i = 0, j = 0, k = 0;
2  while (i < A.length && j < B.length) {
▶    if (A[i] <= B[j])
▶      C[k++] = A[i++];
5    else
6      C[k++] = B[j++];
7  }
8  while (i < A.length)
9    C[k++] = A[i++];
10 while (j < B.length)
11   C[k++] = B[j++];
```



A[1] = 7 ≤ B[1] = 9 → 7, C'ye yazılır; i ilerler.

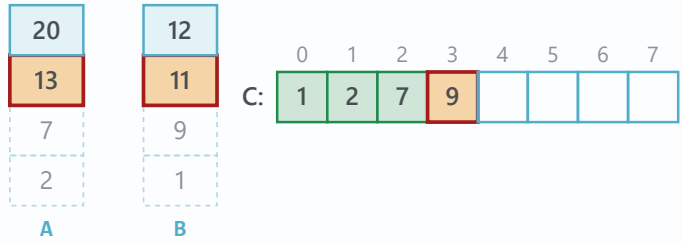
Adım 4 / 9

DEĞİŞKENLER

`i = 2` `j = 1` `k = 3`

Birleştirme İşlemi (Merge): Adım Adım

```
1  int i = 0, j = 0, k = 0;
2  while (i < A.length && j < B.length) {
3    if (A[i] <= B[j])
4      C[k++] = A[i++];
5    else
6      C[k++] = B[j++];
7  }
8  while (i < A.length)
9    C[k++] = A[i++];
10 while (j < B.length)
11   C[k++] = B[j++];
```



A[2] = 13 > B[1] = 9 → 9, C'ye yazılır; j ilerler.

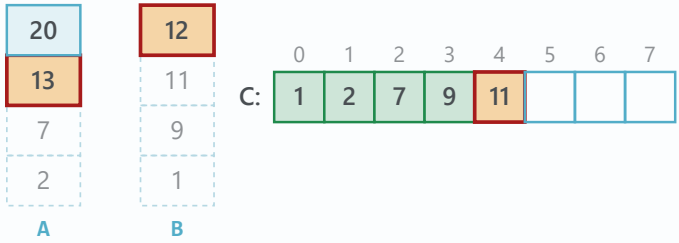
Adım 5 / 9

DEĞİŞKENLER

i = 2 j = 2 k = 4

Birleştirme İşlemi (Merge): Adım Adım

```
1  int i = 0, j = 0, k = 0;
2  while (i < A.length && j < B.length) {
3    if (A[i] <= B[j])
4      C[k++] = A[i++];
▶    else
▶      C[k++] = B[j++];
7  }
8  while (i < A.length)
9    C[k++] = A[i++];
10 while (j < B.length)
11   C[k++] = B[j++];
```



A[2] = 13 > B[2] = 11 → 11, C'ye yazılır; j ilerler.

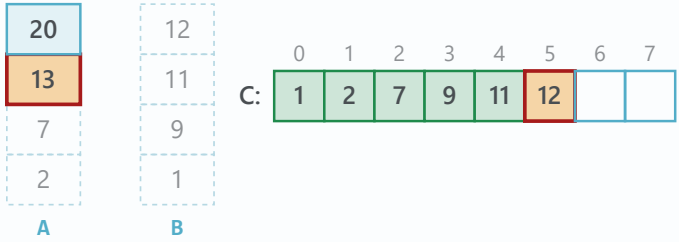
Adım 6 / 9

DEĞİŞKENLER

`i = 2` `j = 3` `k = 5`

Birleştirme İşlemi (Merge): Adım Adım

```
1  int i = 0, j = 0, k = 0;
2  while (i < A.length && j < B.length) {
3    if (A[i] <= B[j])
4      C[k++] = A[i++];
▶    else
▶      C[k++] = B[j++];
7  }
8  while (i < A.length)
9    C[k++] = A[i++];
10 while (j < B.length)
11   C[k++] = B[j++];
```



A[2] = 13 > B[3] = 12 → 12, C'ye yazılır; j ilerler.

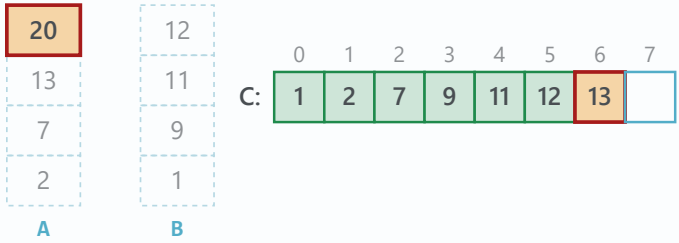
Adım 7 / 9

DEĞİŞKENLER

`i = 2` `j = 4` `k = 6`

Birleştirme İşlemi (Merge): Adım Adım

```
1  int i = 0, j = 0, k = 0;
2  while (i < A.length && j < B.length) {
3    if (A[i] <= B[j])
4      C[k++] = A[i++];
5    else
6      C[k++] = B[j++];
7  }
▶  while (i < A.length)
▶    C[k++] = A[i++];
10 while (j < B.length)
11   C[k++] = B[j++];
```



B bitti: A'da kalan 13 doğrudan C'ye kopyalanır (karşılaştırma yok).

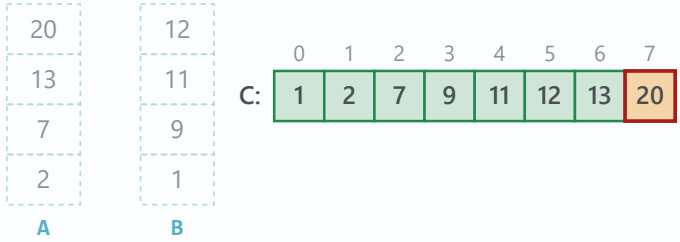
Adım 8 / 9

DEĞİŞKENLER

i = 3 j = 4 k = 7

Birleştirme İşlemi (Merge): Adım Adım

```
1  int i = 0, j = 0, k = 0;
2  while (i < A.length && j < B.length) {
3    if (A[i] <= B[j])
4      C[k++] = A[i++];
5    else
6      C[k++] = B[j++];
7  }
▶  while (i < A.length)
▶    C[k++] = A[i++];
10 while (j < B.length)
11   C[k++] = B[j++];
```



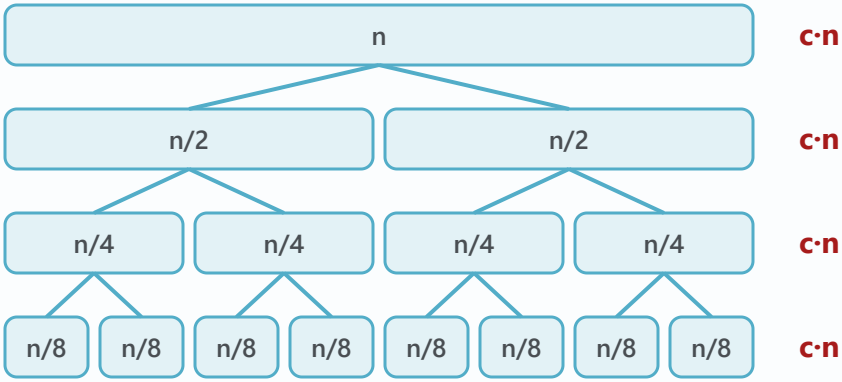
B bitti: A'da kalan 20 doğrudan C'ye kopyalanır (karşılaştırma yok). Sonuç: **1 2 7 9 11 12 13 20**. Her öğe bir kez yazıldı → Θ(n).

Adım 9 / 9

DEĞİŞKENLER

`i = 4` `j = 4` `k = 8`

Birleştirme Sıralaması: Neden Θ(n log n)?

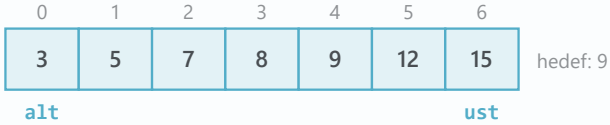


$\log_2 n + 1$ düzey × c·n iş ⇒ T(n) = Θ(n log n)

- n öğeli listeyi birleştirmek **Θ(n)** sürer (yukarıdaki örnekte 8 öğe, 8 yazma, 6 karşılaştırma).
- Her düzeyde alt problemlerin toplam boyutu n'dir → her düzeyin toplam birleştirme işi c·n.
- Boyut her düzeyde yarıya iner: 1'e inmek için **$\log_2 n$** bölme gerekir.
- Eklemeli sıralamanın Θ(n^2) en kötü durumuna karşı, birleştirme sıralaması **her girdide** Θ(n log n)'dir; bedeli O(n) ek bellektir.

İkili Arama: 9'u Bul

```
1  int ikiliArama(int[] dizi, int aranan) {
▶    int alt = 0, ust = dizi.length - 1;
3    while (alt <= ust) {
4      int orta = (alt + ust) / 2;
5      int tahmin = dizi[orta];
6      if (tahmin == aranan)
7        return orta;     // bulundu
8      else if (tahmin < aranan)
9        alt = orta + 1;
10     else
11       ust = orta - 1;
12   }
13   return -1;          // bulunamadı
14 }
```



Örnek: **9'u bul**. Dizi sıralı; aralık tüm dizi: alt = 0, ust = 6.

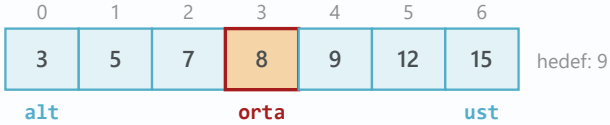
Adım 1 / 7

DEĞİŞKENLER

`alt = 0` `ust = 6` `orta = –` `tahmin = –`

İkili Arama: 9'u Bul

```
1  int ikiliArama(int[] dizi, int aranan) {
2    int alt = 0, ust = dizi.length - 1;
▶    while (alt <= ust) {
▶      int orta = (alt + ust) / 2;
▶      int tahmin = dizi[orta];
6      if (tahmin == aranan)
7        return orta;     // bulundu
8      else if (tahmin < aranan)
9        alt = orta + 1;
10     else
11       ust = orta - 1;
12   }
13   return -1;          // bulunamadı
14 }
```



`orta = (0 + 6) / 2 = 3`, `tahmin = dizi[3] = 8`.

Adım 2 / 7

DEĞİŞKENLER

`alt = 0` `ust = 6` `orta = 3` `tahmin = 8`

İkili Arama: 9'u Bul

```
1  int ikiliArama(int[] dizi, int aranan) {
2    int alt = 0, ust = dizi.length - 1;
3    while (alt <= ust) {
4      int orta = (alt + ust) / 2;
5      int tahmin = dizi[orta];
6      if (tahmin == aranan)
7        return orta;     // bulundu
▶      else if (tahmin < aranan)
▶        alt = orta + 1;
10     else
11       ust = orta - 1;
12   }
13   return -1;          // bulunamadı
14 }
```



8 < 9 → hedef sağ yarıda: `alt = 4`. Sol yarı elendi.

Adım 3 / 7

DEĞİŞKENLER

`alt = 4` `ust = 6` `orta = 3` `tahmin = 8`

İkili Arama: 9'u Bul

```
1  int ikiliArama(int[] dizi, int aranan) {
2    int alt = 0, ust = dizi.length - 1;
▶    while (alt <= ust) {
▶      int orta = (alt + ust) / 2;
▶      int tahmin = dizi[orta];
6      if (tahmin == aranan)
7        return orta;     // bulundu
8      else if (tahmin < aranan)
9        alt = orta + 1;
10     else
11       ust = orta - 1;
12   }
13   return -1;          // bulunamadı
14 }
```



`orta = (4 + 6) / 2 = 5`, `tahmin = dizi[5] = 12`.

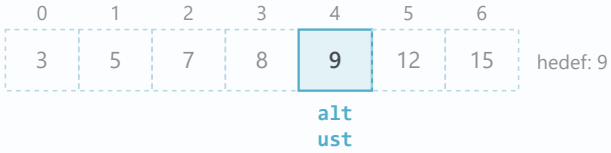
Adım 4 / 7

DEĞİŞKENLER

`alt = 4` `ust = 6` `orta = 5` `tahmin = 12`

İkili Arama: 9'u Bul

```
1  int ikiliArama(int[] dizi, int aranan) {
2    int alt = 0, ust = dizi.length - 1;
3    while (alt <= ust) {
4      int orta = (alt + ust) / 2;
5      int tahmin = dizi[orta];
6      if (tahmin == aranan)
7        return orta;     // bulundu
8      else if (tahmin < aranan)
9        alt = orta + 1;
▶      else
▶        ust = orta - 1;
12   }
13   return -1;          // bulunamadı
14 }
```



12 > 9 → hedef sol yarıda: `ust = 4`. Sağ taraf elendi.

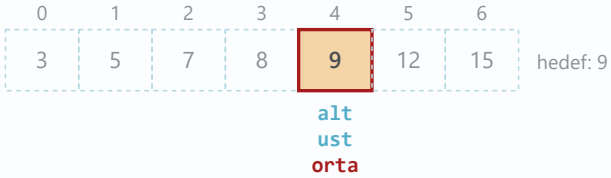
Adım 5 / 7

DEĞİŞKENLER

`alt = 4` `ust = 4` `orta = 5` `tahmin = 12`

İkili Arama: 9'u Bul

```
1  int ikiliArama(int[] dizi, int aranan) {
2    int alt = 0, ust = dizi.length - 1;
▶    while (alt <= ust) {
▶      int orta = (alt + ust) / 2;
▶      int tahmin = dizi[orta];
6      if (tahmin == aranan)
7        return orta;     // bulundu
8      else if (tahmin < aranan)
9        alt = orta + 1;
10     else
11       ust = orta - 1;
12   }
13   return -1;          // bulunamadı
14 }
```



`orta = (4 + 4) / 2 = 4`, `tahmin = dizi[4] = 9`.

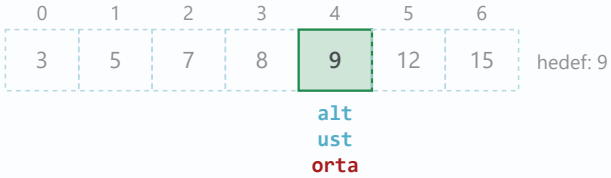
Adım 6 / 7

DEĞİŞKENLER

`alt = 4` `ust = 4` `orta = 4` `tahmin = 9`

İkili Arama: 9'u Bul

```
1  int ikiliArama(int[] dizi, int aranan) {
2    int alt = 0, ust = dizi.length - 1;
3    while (alt <= ust) {
4      int orta = (alt + ust) / 2;
5      int tahmin = dizi[orta];
▶      if (tahmin == aranan)
▶        return orta;     // bulundu
8      else if (tahmin < aranan)
9        alt = orta + 1;
10     else
11       ust = orta - 1;
12   }
13   return -1;          // bulunamadı
14 }
```



tahmin == aranan → **return 4**. Yalnızca 3 karşılaştırma ($\lfloor \log_2 7 \rfloor + 1 = 3$).

Adım 7 / 7

DEĞİŞKENLER

`alt = 4` `ust = 4` `orta = 4` `tahmin = 9`

İkili Arama: Böl ve Fethet Analizi

İkili arama (Binary Search): sıralı listede bir öğeyi arar	**Maliyet**
Böl (Divide): orta öğeyi kontrol et.	Θ(1)
Fethet (Conquer): seçilen alt diziyi özyinelemeli olarak ara.	T(n/2)
Birleştir (Combine): önemsiz (trivial); alt dizinin cevabı asıl cevaptır.	Θ(1)

$$T(n) = 1 \cdot T(n/2) + \Theta(1)$$

alt problem sayısı

alt problem büyüklüğü

bölme + birleştirme

Açılım: $T(n) = T(n/2) + c = T(n/4) + 2c = \ldots = T(1) + c \cdot \log_2 n \Rightarrow \mathbf{T(n) = \Theta(\log n)}$

Bir Sayının Üssünü Alma: a^n

Problem: a^n'i hesapla ($n \in \mathbb{N}$). **Doğal algoritma:** $a^n = a \cdot a \cdot \ldots \cdot a \rightarrow \Theta(n)$ çarpma.

Böl ve fethet:

$$a^n = a^{n/2} \cdot a^{n/2} \quad (n \text{ çift ise})$$
$$a^n = a^{(n-1)/2} \cdot a^{(n-1)/2} \cdot a \quad (n \text{ tek ise})$$

```
long us(long a, int n) {
  if (n == 0) return 1;
  long yarim = us(a, n / 2);        // a^(n/2), tek kez!
  if (n % 2 == 0)
    return yarim * yarim;           // n çift
  else
    return a * yarim * yarim;       // n tek
}
```

n	doğal	böl ve fethet
10	10	6
62	62	11
1.000	1.000	16
1.000.000	1.000.000	27

çarpma sayıları (yukarıdaki kodla). Örn. 2^{10}: $n = 10 \rightarrow 5 \rightarrow 2 \rightarrow 1 \rightarrow 0$.

$T(n) = T(n/2) + \Theta(1)$
$\Rightarrow$ **$T(n) = \Theta(\lg n)$**

us(2, 10): Adım Adım

```
1 long us(long a, int n) {
▶   if (n == 0) return 1;
▶   long yarim = us(a, n / 2);
4   if (n % 2 == 0)
5     return yarim * yarim;
6   else
7     return a * yarim * yarim;
8 }
```

ÇAĞRI YIĞINI

`us(2, 10)`



us(2, n): her çağrı n'yi yarıya indirir

`us(2, 10)` : n ≠ 0. Önce `yarim = us(2, 5)` hesaplanmalı (10 / 2 = 5); us(2, 10) yığında bekler.

Adım 1 / 9

DEĞİŞKENLER

`a = 2` `n = 10` `yarim = –` `sonuç = –` `çarpma = 0`

us(2, 10): Adım Adım

```
1 long us(long a, int n) {
▶   if (n == 0) return 1;
▶   long yarim = us(a, n / 2);
4   if (n % 2 == 0)
5     return yarim * yarim;
6   else
7     return a * yarim * yarim;
8 }
```



us(2, n): her çağrı n'yi yarıya indirir

`us(2, 5)` : n ≠ 0 → `yarim = us(2, 2)` çağrılır (5 / 2 = 2, tam sayı bölmesi).

Adım 2 / 9

ÇAĞRI YIĞINI

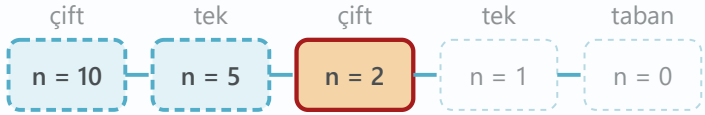
- **us(2, 5)**
- us(2, 10)

DEĞİŞKENLER

`a = 2` `n = 5` `yarim = –` `sonuç = –` `çarpma = 0`

us(2, 10): Adım Adım

```
1 long us(long a, int n) {
▶   if (n == 0) return 1;
▶   long yarim = us(a, n / 2);
4   if (n % 2 == 0)
5     return yarim * yarim;
6   else
7     return a * yarim * yarim;
8 }
```



us(2, n): her çağrı n'yi yarıya indirir

`us(2, 2)` : n ≠ 0 → `yarim = us(2, 1)` çağrılır (2 / 2 = 1, tam sayı bölmesi).

ÇAĞRI YIĞINI

- **us(2, 2)**
- us(2, 5)
- us(2, 10)

Adım 3 / 9

DEĞİŞKENLER

`a = 2` `n = 2` `yarim = –` `sonuç = –` `çarpma = 0`

us(2, 10): Adım Adım

```
1 long us(long a, int n) {
  if (n == 0) return 1;
  long yarim = us(a, n / 2);
4   if (n % 2 == 0)
5     return yarim * yarim;
6   else
7     return a * yarim * yarim;
8 }
```

ÇAĞRI YIĞINI

- **us(2, 1)**
- us(2, 2)
- us(2, 5)
- us(2, 10)



us(2, n): her çağrı n'yi yarıya indirir

`us(2, 1)`: n ≠ 0 → `yarim = us(2, 0)` çağrılır (1 / 2 = 0, tam sayı bölmesi).

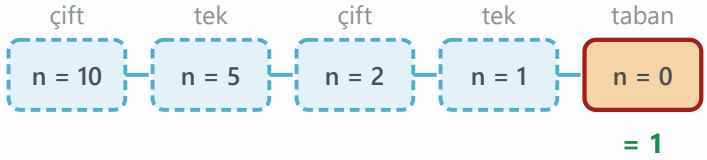
Adım 4 / 9

DEĞİŞKENLER

`a = 2` `n = 1` `yarim = –` `sonuç = –` `çarpma = 0`

us(2, 10): Adım Adım

```
1 long us(long a, int n) {
  if (n == 0) return 1;
3   long yarim = us(a, n / 2);
4   if (n % 2 == 0)
5     return yarim * yarim;
6   else
7     return a * yarim * yarim;
8 }
```



us(2, n): her çağrı n'yi yarıya indirir

n == 0: taban durum, **1** döner. Yığında 5 çerçeve var; n her çağrıda yarıya indiği için derinlik ≈ $\log_2 n$.

ÇAĞRI YIĞINI

- **us(2, 0)**
- us(2, 1)
- us(2, 2)
- us(2, 5)
- us(2, 10)

Adım 5 / 9

DEĞİŞKENLER

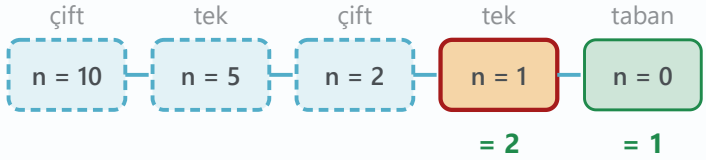
`a = 2` `n = 0` `yarim = –` `sonuç = 1` `çarpma = 0`

us(2, 10): Adım Adım

```
1 long us(long a, int n) {
2   if (n == 0) return 1;
3   long yarim = us(a, n / 2);
    if (n % 2 == 0)
5     return yarim * yarim;
6   else
      return a * yarim * yarim;
8 }
```

ÇAĞRI YIĞINI

- us(2, 1)
- us(2, 2)
- us(2, 5)
- us(2, 10)



us(2, n): her çağrı n'yi yarıya indirir

n = 1 tek: `a * yarim * yarim` = 2 · 1 · 1 = **2** (2 çarpma).

Adım 6 / 9

DEĞİŞKENLER

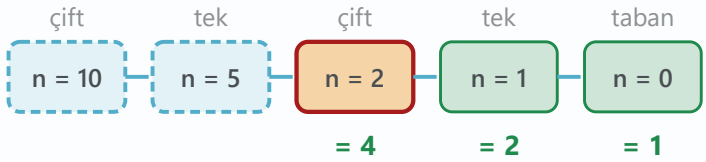
a = **2** | n = **1** | yarim = **1** | sonuç = **2** | çarpma = **2**

us(2, 10): Adım Adım

```
1 long us(long a, int n) {
2   if (n == 0) return 1;
3   long yarim = us(a, n / 2);
▶   if (n % 2 == 0)
▶     return yarim * yarim;
6   else
7     return a * yarim * yarim;
8 }
```

ÇAĞRI YIĞINI

- **us(2, 2)**
- us(2, 5)
- us(2, 10)



us(2, n): her çağrı n'yi yarıya indirir

n = 2 çift: `yarim * yarim` = 2 · 2 = **4** (1 çarpma). `yarim` bir kez hesaplanıp iki kez kullanılır: tek özyinelemeli çağrı yeter.

Adım 7 / 9

DEĞİŞKENLER

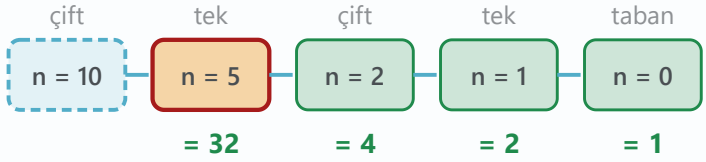
`a = 2` `n = 2` `yarim = 2` `sonuç = 4` `çarpma = 3`

us(2, 10): Adım Adım

```
1 long us(long a, int n) {
2   if (n == 0) return 1;
3   long yarim = us(a, n / 2);
▶   if (n % 2 == 0)
5     return yarim * yarim;
6   else
▶     return a * yarim * yarim;
8 }
```

ÇAĞRI YIĞINI

- **us(2, 5)**
- us(2, 10)



us(2, n): her çağrı n'yi yarıya indirir

n = 5 tek: `a * yarim * yarim` = 2 · 4 · 4 = **32** (2 çarpma).

Adım 8 / 9

DEĞİŞKENLER

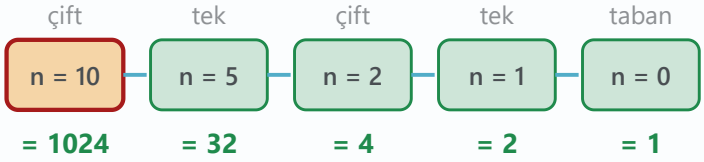
`a = 2` `n = 5` `yarim = 4` `sonuç = 32` `çarpma = 5`

us(2, 10): Adım Adım

```
1 long us(long a, int n) {
2   if (n == 0) return 1;
3   long yarim = us(a, n / 2);
▶   if (n % 2 == 0)
▶     return yarim * yarim;
6   else
7     return a * yarim * yarim;
8 }
```

ÇAĞRI YIĞINI

`us(2, 10)`



us(2, n): her çağrı n'yi yarıya indirir

n = 10 çift: `yarim * yarim` = 32 · 32 = **1024** (1 çarpma). Toplam **6 çarpma**; doğal yöntem 10 çarpma ister.

Adım 9 / 9

DEĞİŞKENLER

`a = 2` `n = 10` `yarim = 32` `sonuç = 1024` `çarpma = 6`

Öklid Algoritması (EBOB)

- **En büyük ortak böleni** (EBOB, gcd) bulur.
- Bilinen en eski algoritmalardan biridir (Öklid, MÖ ~300); en basit EBOB yöntemi.
- Temel fikir: **ebob(a, b) = ebob(b, a mod b)**, ebob(a, 0) = a.

1. **Adım 1:** büyük sayıyı küçüğe böl, kalanı al.
2. **Adım 2:** kalan sıfır olana kadar (bölen, kalan) çiftiyle tekrarla.
3. **Adım 3:** kalan sıfır olduğunda son **bölen** EBOB'dur.

```
int ebob(int a, int b) {
  if (b == 0)
    return a;
  return ebob(b, a % b);
}
```

Neden doğru?

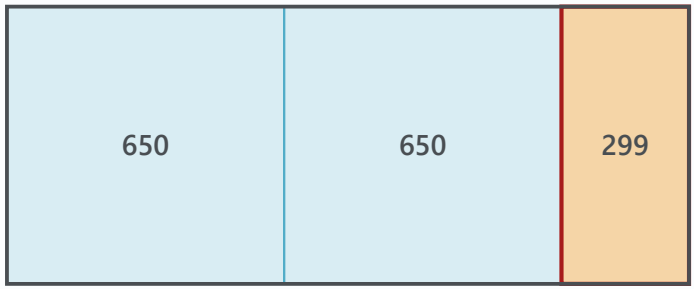
a = q·b + r ise a ile b'yi bölen her sayı r = a − q·b'yi de böler; b ile r'yi bölen her sayı a'yı da böler. Ortak bölenler aynıdır.

ebob(1599, 650): Adım Adım

```
1 int ebob(int a, int b) {
▶   if (b == 0)
3     return a;
▶   return ebob(b, a % b);
5 }
```

ÇAĞRI YIĞINI

`ebob(1599, 650)`



1599 × 650 dikdörtgen

1599 = 2 · 650 + **299**. 1599×650 dikdörtgenden 2 kare kesilir, kalan şerit 650×299 → `ebob(650, 299)`.

Adım 1 / 7

DEĞİŞKENLER

`a = 1599` `b = 650` `a%b = 299`

ebob(1599, 650): Adım Adım

```
1 int ebob(int a, int b) {
▶   if (b == 0)
3     return a;
▶   return ebob(b, a % b);
5 }
```

ÇAĞRI YIĞINI

- **ebob(650, 299)**
- ebob(1599, 650)



650 × 299 dikdörtgen

650 = 2 · 299 + **52**. 650×299 dikdörtgenden 2 kare kesilir, kalan şerit 299×52 → `ebob(299, 52)`.

Adım 2 / 7

DEĞİŞKENLER

`a = 650` `b = 299` `a%b = 52`

ebob(1599, 650): Adım Adım

```
1 int ebob(int a, int b) {
▶   if (b == 0)
3     return a;
▶   return ebob(b, a % b);
5 }
```

ÇAĞRI YIĞINI

- **ebob(299, 52)**
- ebob(650, 299)
- ebob(1599, 650)



299 × 52 dikdörtgen

299 = 5 · 52 + **39**. 299×52 dikdörtgenden 5 kare kesilir, kalan şerit 52×39 → `ebob(52, 39)`.

Adım 3 / 7

DEĞİŞKENLER

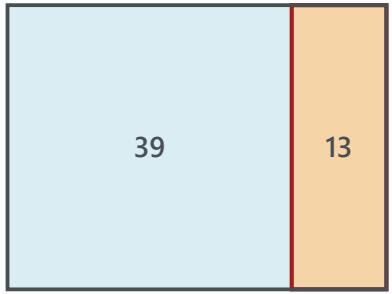
`a = 299` `b = 52` `a%b = 39`

ebob(1599, 650): Adım Adım

```
1 int ebob(int a, int b) {
▶   if (b == 0)
3     return a;
▶   return ebob(b, a % b);
5 }
```

ÇAĞRI YIĞINI

- **ebob(52, 39)**
- ebob(299, 52)
- ebob(650, 299)
- ebob(1599, 650)



52 × 39 dikdörtgen

52 = 1 · 39 + **13**. 52×39 dikdörtgenden 1 kare kesilir, kalan şerit 39×13 → `ebob(39, 13)`.

Adım 4 / 7

DEĞİŞKENLER

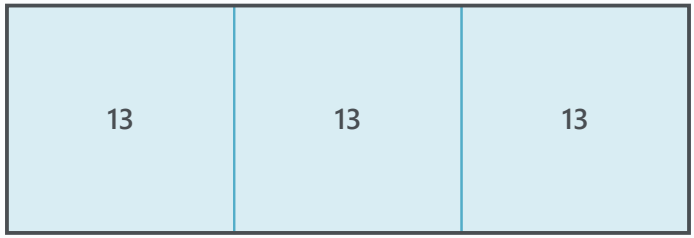
`a = 52` `b = 39` `a%b = 13`

ebob(1599, 650): Adım Adım

```
1 int ebob(int a, int b) {
▶   if (b == 0)
3     return a;
▶   return ebob(b, a % b);
5 }
```

ÇAĞRI YIĞINI

- **ebob(39, 13)**
- ebob(52, 39)
- ebob(299, 52)
- ebob(650, 299)
- ebob(1599, 650)



39 × 13 dikdörtgen

39 = 3 · 13 + **0**. 39×13 dikdörtgenden 3 kare kesilir, kalan şerit 13×0 → `ebob(13, 0)`.

Adım 5 / 7

DEĞİŞKENLER

`a = 39` `b = 13` `a%b = 0`

ebob(1599, 650): Adım Adım

```
1 int ebob(int a, int b) {
▶   if (b == 0)
▶     return a;
4   return ebob(b, a % b);
5 }
```

ÇAĞRI YIĞINI

- **ebob(13, 0)**
- ebob(39, 13)
- ebob(52, 39)
- ebob(299, 52)
- ebob(650, 299)
- ebob(1599, 650)

ebob(13, 0) = **13**

b == 0 → **return 13**. 13, 1599 ile 650'yi tam bölen en büyük sayıdır.

Adım 6 / 7

DEĞİŞKENLER

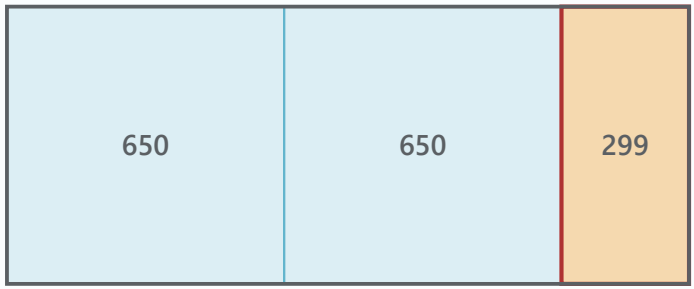
a = **13** b = **0**

ebob(1599, 650): Adım Adım

```
1 int ebob(int a, int b) {
2   if (b == 0)
3     return a;
  ▶ return ebob(b, a % b);
5 }
```

ÇAĞRI YIĞINI

`ebob(1599, 650)`



1599 × 650 dikdörtgen

Dönüş değeri 13 tüm çağrılardan aynen geri döner: **ebob(1599, 650) = 13**. 5 bölme yetti. 13 × 13'lük kareler 1599 × 650'yi tam döşer.

Adım 7 / 7

DEĞİŞKENLER

`a = 1599` `b = 650` `sonuç = 13`

Öklid Algoritması: Neden Hızlı?

- Verimli ve hızlı çalışır: **her iki adımda** büyük sayı en az **yarıya** iner (a mod b < a/2).
- Bu yüzden adım sayısı en fazla ≈ 2 $\log_2$ min(a, b): **O(log min(a, b))**.
- **En kötü senaryo** ardışık Fibonacci sayılarıyla oluşur: her bölümde q = 1 çıkar, sayılar en yavaş küçülür (Lamé teoremi).
- Karşılaştırma: 1'den min(a, b)'ye kadar tüm bölenleri denemek O(min(a, b)) sürer.

En kötü örnek: ebob(55, 34)

```
55 = 1 · 34 + 21
34 = 1 · 21 + 13
21 = 1 · 13 + 8
13 = 1 · 8 + 5
8 = 1 · 5 + 3
5 = 1 · 3 + 2
3 = 1 · 2 + 1
2 = 2 · 1 + 0
```

8 bölme, EBOB = 1. Karşılaştırın: ebob(1599, 650) yalnızca 5 bölme.

Özet

Konu	Hatırlanacaklar
Algoritma	girdi → açık, sonlu, etkin adımlar → çıktı; dilden bağımsızdır
Analiz	T(n): girdi boyutuna göre temel işlem sayısı; zaman ve bellek
Durumlar	en kötü (garanti), ortalama (beklenen), en iyi (yanıltıcı)
Gösterimler	O üst, Ω alt, Θ sıkı sınır; sabitler ve düşük terimler atılır
Sınıflar	$1 < \log n < \sqrt{n} < n < n \log n < n^2 < n^3 < 2^n < n!$
Eklemeli sıralama	en iyi $\Theta(n)$, en kötü $\Theta(n^2)$, yerinde, $O(1)$ ek bellek
Böl ve fethet	birleştirme sıralaması $2T(n/2) + \Theta(n) = \Theta(n \log n)$; ikili arama ve üs alma $T(n/2) + \Theta(1) = \Theta(\log n)$
Öklid	ebob(a, b) = ebob(b, a mod b); $O(\log \min(a, b))$; en kötü: ardışık Fibonacci

ÖZET

Etkileşimli Simülatörler

Algoritma Karmaşıklığı Analizi

O(log n), O(n), O(n log n), $O(n^2)$ ve $O(n^3)$ örnek JavaScript fonksiyonları ya da yeni bir fonksiyon; beş farklı girdi boyutunda tarayıcıda ölçülen sürelerin grafikte karşılaştırması.

Büyüme Hesaplayıcı

Verilen bir n için 1'den n!'e kadar her fonksiyonun işlem sayısını ve saniyede 10^9 işlem yapan bir bilgisayardaki süresini gösterir.

ÖZET

Sık Yapılan Hatalar

O = en kötü, Ω = en iyi sanmak

O, Ω, Θ sınır türleridir; her durum (en kötü, en iyi) için ayrı ayrı yazılabilir.

Yanlış Θ çıkarımı

En kötü $O(n^2)$, en iyi $\Omega(n)$ ise tüm girdiler için tek bir Θ yoktur. Θ için iki sınır aynı olmalı.

Süreyi saniyeyle ölçmek

Saniye makineye bağlıdır; analizde n'ye bağlı işlem sayısı ve büyüme hızı kullanılır.

Ezberlemeyi unutmak

Saf özyinelemeli Fibonacci $O(2^n)$'dir; aynı alt problemi saklamak onu $O(n)$'e indirir.

Ortalama = garanti sanmak

Hızlı sıralama ortalamada $O(n \log n)$, ama sıralı girdide ilk öğe pivotla $O(n^2)$'dir.

Sıralanmamış diziye ikili arama

İkili arama yalnızca sıralı dizide doğrudur; aksi hâlde var olan öğe "yok" bulunabilir.

ÖZET

Örnek Problemler

1. Eklemeli sıralama [5, 4, 3, 2, 1] ve [1, 2, 3, 4, 5] dizilerinde kaç karşılaştırma ve kaç kaydırma yapar?
2. $T(n) = 5n^2 + 100n \log_2 n + 7$ için Θ sınıfını yazın. $n = 1000$'de hangi terim baskındır?
3. [2, 4, 7, 10, 13, 18, 21, 25, 30] dizisinde ikili arama ile 21 ve 5'i arayın; incelenen orta öğeleri yazın.
4. ebob(252, 105)'i Öklid algoritmasıyla bulun.
5. 2^{13} yukarıdaki `us` fonksiyonuyla hesaplanırken hangi üsler için çağrı yapılır, kaç çarpma gerekir?

▶ **Cevapları göster**

ÖZET

Alıştırmalar

1. Bir dizide verilen bir sayının kaç kez geçtiğini bulan algoritmayı yazın; zaman ve alan karmaşıklığını verin.
2. İç içe döngüler: `for i in 0..n-1: for j in i..n-1: işlem` kaç kez çalışır? Sınıfı nedir?
3. Eklemeli sıralamayı azalan sırada sıralayacak şekilde değiştirin. Hangi satır değişir?
4. Öklid algoritmasını döngüyle (özyinelemesiz) yazın.
5. Saniyede 10^9 işlemle 1 saniyede çözülebilecek en büyük n: $f(n) = n^2$ ve $f(n) = 2^n$ için bulun.
6. `while (n > 1) n = n / 2;` döngüsü kaç kez döner?

▶ **Cevaplar / ipuçları**

Kaynaklar

- Cormen, Leiserson, Rivest ve Stein, *Introduction to Algorithms*, 4. baskı, MIT Press, 2022 (Bölüm 1–3).
- Sedgewick ve Wayne, *Algorithms*, 4. baskı, Addison-Wesley, 2011 (Bölüm 1.4: Analysis of Algorithms).
- D. E. Knuth, *The Art of Computer Programming, Vol. 1: Fundamental Algorithms*, 3. baskı, 1997 (algoritmanın özellikleri; Öklid).
- J. Kleinberg ve É. Tardos, *Algorithm Design*, Pearson, 2005 (Bölüm 2: Basics of Algorithm Analysis).

Son

Bölüm 1: Giriş

Sonraki bölüm: Bölüm 1.2: Algoritma Karmaşıklığı · Terim sözlüğü